

NoSQL databases

NOT ONLY SQL

What NoSQL means

The term has been first used in '98 by Carlo Strozzi

- It referred to an open-source RDBMS that used a query language different from SQL

In 2009 it was adopted by a meetup in San Francisco

- Goal: discuss open-source projects related to the newest databases from Google and Amazon
- Participants: Voldemort, Cassandra, Dynomite, HBase, Hypertable, CouchDB, MongoDB

Today, **NoSQL** indicates DBMSs adopting a different data model from the relational one

- NoSQL = Not Only SQL
- According to Strozzi himself, NoREL would have been a more proper noun

NoSQL common features

Not just rows and tables

- Several data model adopted to store and manipulate data
- **Solve the Impedance mismatch**
 - Data are stored according to the relational model, but applications to modify them typically rely on the object-oriented model: objects or class definitions must be mapped to database tables defined by a relational schema but the two models has different expressivity, different constraints, ecc..
- **Models Aggregates** of concepts rather than single concepts

Freedom from rigid schemas

- Data can be stored or queried without pre-defining a schema (***schemaless*** or ***soft-schema***)

Freedom from joins

- Joins are either not supported or discouraged

Distributed, shared-nothing architecture (run on clusters!)

- **Trivial scalability** in a distributed environment with no performance decay
- Each workstation uses its own disks and RAM

Why we need NoSQL?

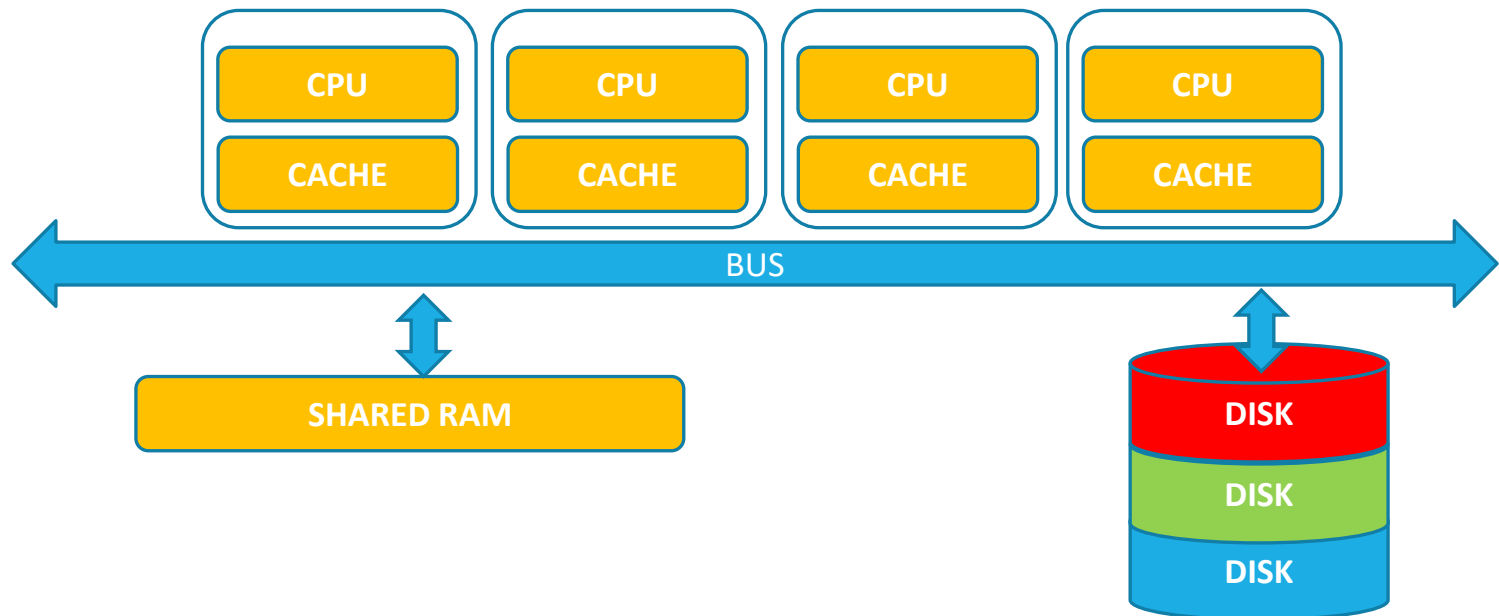
The 2000s did see **several large web properties dramatically increase in scale!**

- Websites started tracking activity and structure in a very detailed way. Large sets of data appeared: links, social networks, activity in logs, mapping data. With this growth in data came a growth in users
- Coping with the increase in data and traffic required more computing resources
- **Scaling up implies bigger machines, more processors, disk storage, and memory.** But bigger machines get more and more expensive, not to mention that there are real limits as your size increases
- **The alternative is to scale out, i.e., use lots of small machines in a cluster.** A cluster of small machines can use commodity hardware and ends up being cheaper at these kinds of scales.
- It can also be **more resilient** while individual machine failures are common, the overall cluster can be built to keep going despite such failures, providing high reliability
- As large properties moved towards clusters, that revealed a new problem: **relational databases are not designed to be run on clusters!**

SMP Architecture

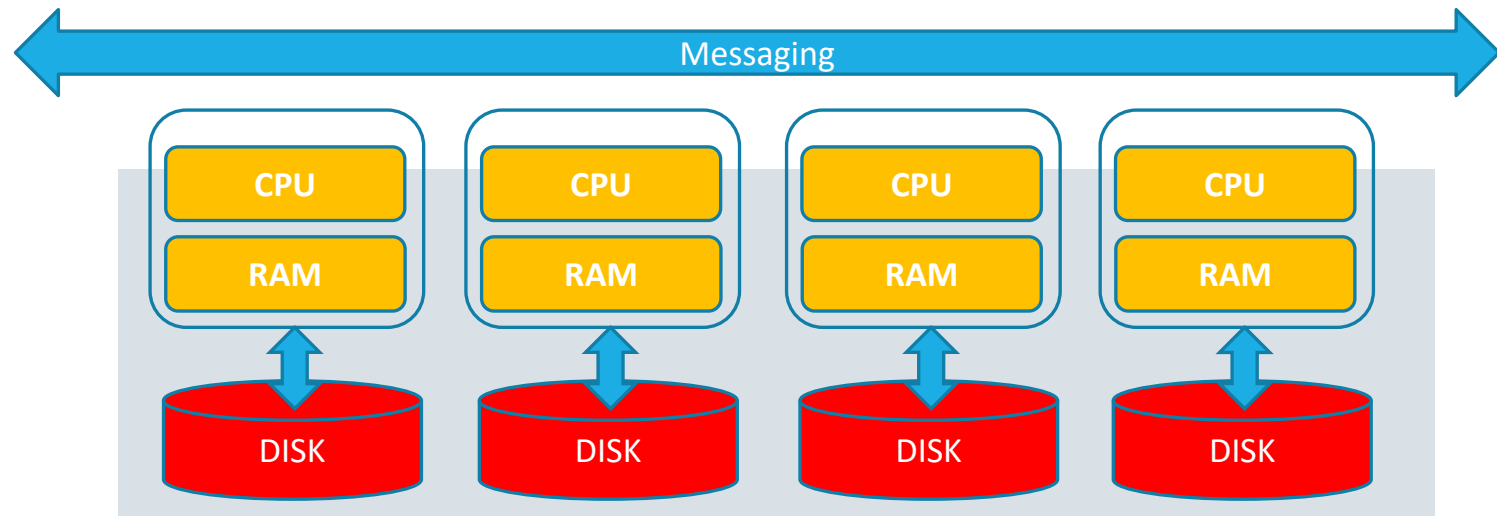
Traditional RDBMS are based on *Symmetric Multi Processing* (SMP) architecture: several processors share the same RAM, the same I/O bus and the same disk/disk array

- The limit of such architecture depends on the physical number of devices that can be mounted and by the BUS bottleneck



MPP Architecture

In a *Massively Parallel Processing* (MPP) Architecture several processors, each equipped with its own RAM and disks, collaborate to solve a **single problem** by splitting it in **several independent tasks**. It is also called **shared nothing architecture**



Scalability

Scalability is the ability of a system, network, or process to handle a growing amount of work in a capable manner or its ability to be enlarged to accommodate that growth.

Methods of adding more resources for a particular application fall into two broad categories: horizontal and vertical scaling.

- To **scale horizontally** (or **scale out**) means to add more nodes to a system, such as adding a new computer to a distributed software application.
- To **scale vertically** (or **scale up**) means to add resources to a single node in a system, typically involving the addition of CPUs or memory to a single computer.

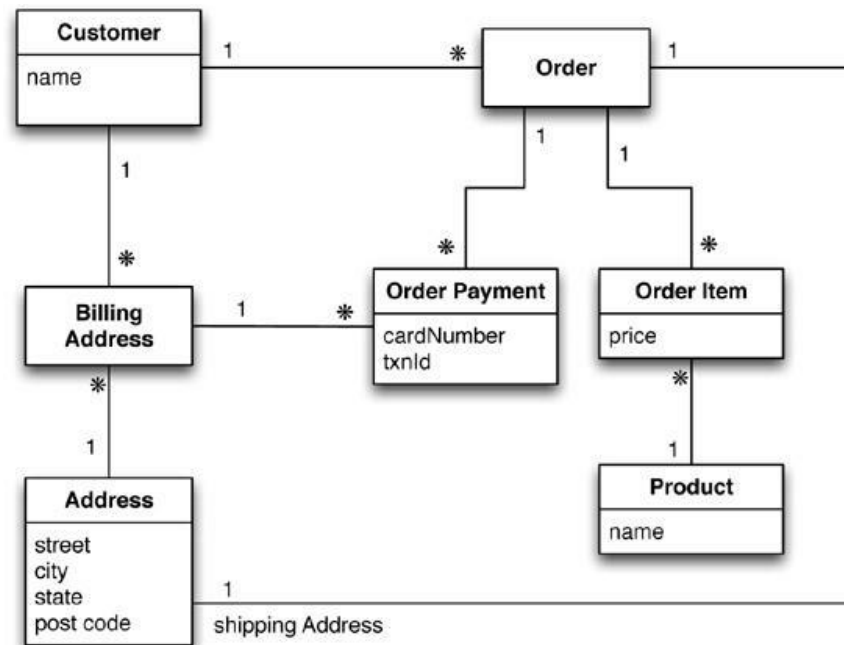
Vertical scalability is typically limited by the actual capability of the architecture to mount more resources and by the physical limitation of such resources (e.g. processor speeds). Nowadays, the availability of low cost commodity hardware and network connections make scaling out often more convenient than scale up.

Aggregate modeling

- The **relational model** divides the information that we want to store into **tuples** (rows): this is a very simple structure for data (which somehow is the key of the success of the relational model and the cause of the relational dominance we experienced from the late 70s to the first years of 2000s)
- **Aggregate orientation** takes a different approach. It recognizes that often you want to operate on data in units that have a more complex structure.
 - It can be handy to think in terms of a complex record that allows lists and other record structures to be nested inside it thus solving the impedance mismatch problem
- As well see, key-value, document, and column-family databases all make use of this more complex record.

Aggregate modeling: an example

Typical use case: customers, orders and products



Aggregate modeling: an example

Relational modeling

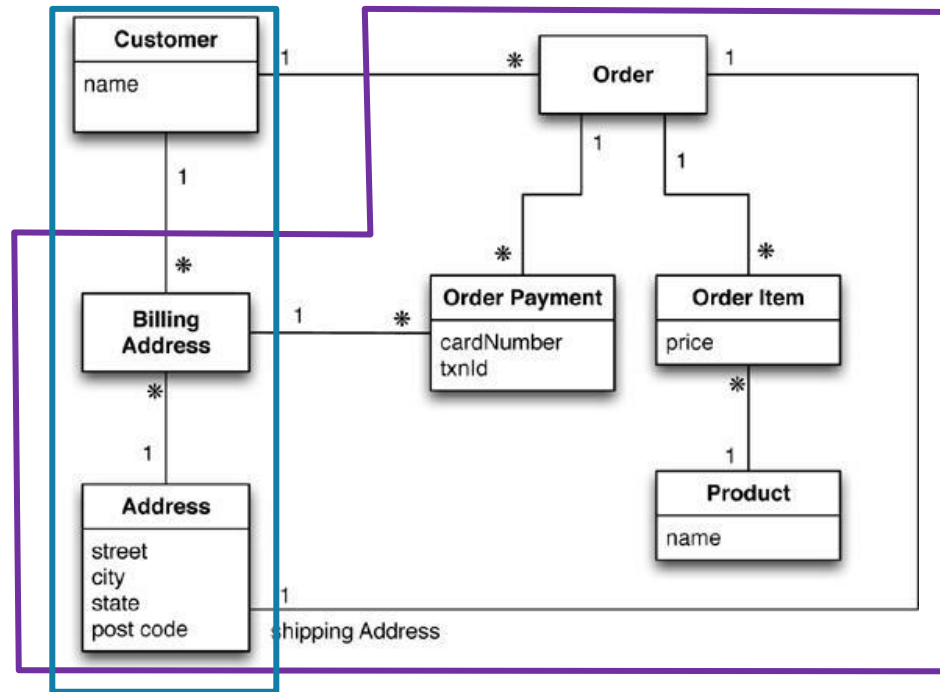
Customer	
Id	Name
1	Martin

Orders		
Id	CustomerId	ShippingAddressId
99	1	77

Product				
Id	Name			
27	NoSQL Distilled			
BillingAddress				
Id	CustomerId	AddressId		
55	1	77		
OrderItem				
Id	OrderId	ProductId	Price	
100	99	27	32.45	
Address				
Id	City			
77	Chicago			
OrderPayment				
Id	OrderId	CardNumber	BillingAddressId	txnId
33	99	1000-1000	55	abelif879rft

Aggregate modeling: an example

One possible aggregate modeling



Aggregate modeling: an example

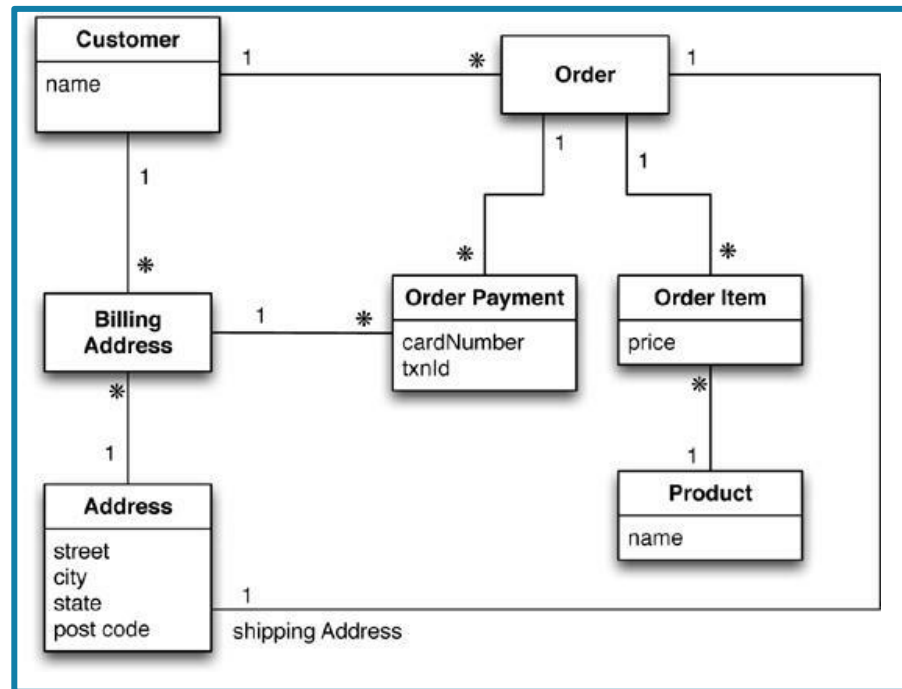
Sample instances (in a document database)

```
// in customers
{
  "id":1,
  "name":"Martin",
  "billingAddress":[{"city":"Chicago"}]
}
```

```
// in orders
{
  "id":99,
  "customerId":1,
  "orderItems":[
    {
      "productId":27,
      "price": 32.45,
      "productName": "NoSQL Distilled"
    }
  ],
  "shippingAddress":[{"city":"Chicago"}]
  "orderPayment":[
    {
      "ccinfo":"1000-1000-1000-1000",
      "txnId":"abelif879rft",
      "billingAddress": {"city": "Chicago"}
    }
  ],
}
```

Aggregate modeling: an example

Another possible aggregate modeling



Aggregate modeling: an example

Sample instances (in a document database)

```
// in customers
{
  "customer": {
    "id": 1,
    "name": "Martin",
    "billingAddress": [{"city": "Chicago"}],
    "orders": [
      {
        "id": 99,
        "customerId": 1,
        "orderItems": [
          {
            "productId": 27,
            "price": 32.45,
            "productName": "NoSQL Distilled"
          }
        ]
      },
      {
        "id": 100,
        "customerId": 1,
        "orderItems": [
          {
            "productId": 28,
            "price": 19.99,
            "productName": "NoSQL Distilled"
          }
        ]
      }
    ],
    "shippingAddress": [{"city": "Chicago"}]
  }
}
```

Aggregate modeling strategy

The *aggregate* term comes from Domain-Driven Design

- An aggregate is a group of tightly coupled objects to be handled as a block
- Aggregates are the basic unit for data manipulation and consistency management

Advantages

- Can be distributed trivially
 - Data that should be used together (e.g., orders and details) are stored together
 - NoSQL DBMSes guarantee consistency at the aggregate level
- Facilitate the developer's job
 - By surpassing the impedance mismatch problem

Disadvantages

- No standard design strategy exists for aggregates
- Can optimize only a limited set of queries

RDBMS are agnostic from applications since they model the data reality and not the way they are accessed by applications

Application databases

With RDBMS and *data modeling* multiple applications store their data in a common, integrated database. This improves communication because all the applications are operating on a consistent set of persistent data

There are downsides to shared database integration

- A structure that is designed to integrate many applications ends up being more complex
- Changes to schema and data by different applications need to be coordinated different applications have different performance needs, thus call for different index structures
- Complex access control policies

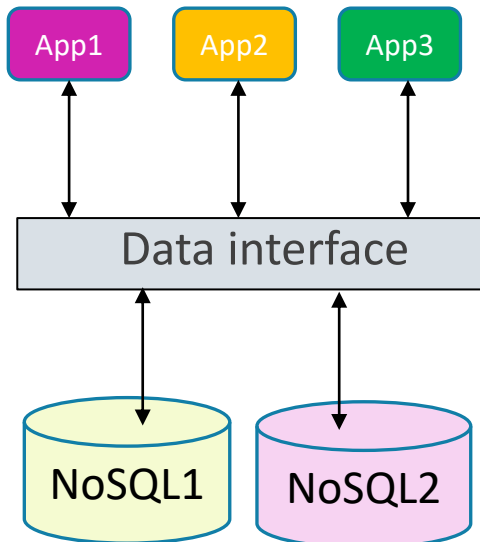
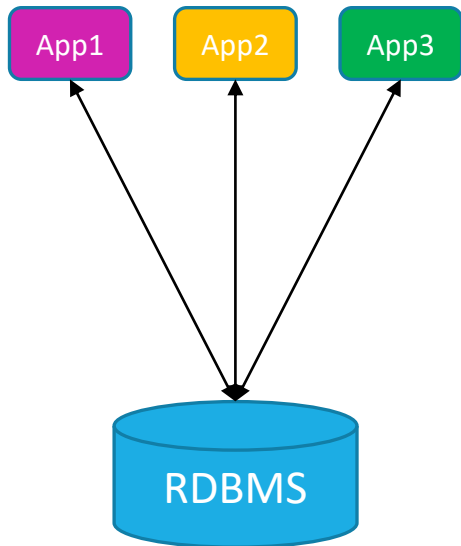
A different approach is to treat your database as an **application database**

Application databases

- An application database is a database portion directly accessed by a single application, which makes it much easier to maintain and evolve
- Interoperability concerns can now shift to the data interfaces of the application (i.e. web services, micro-services)
- Since using application databases there is a decoupling between your internal data representation and the services with which you talk to the outside world, the outside world doesn't have to care how you store your data, allowing you to consider non-relational options
- Furthermore, there are many features of relational databases, such as security, that are less useful to an application database because they can be done by the enclosing application instead

On the other hand, when each application has its own database, the risk arises that data may become non-aligned, mutually inconsistent, and difficult to access in an integrated fashion. All these aspects should be indeed managed at the application level, which does not always take care of them.

Application databases



Strengths of RDBMSs

ACID properties

- Provides guarantees in terms of consistency and concurrent accesses

Data Integration and Normalization of schemas

- Several application can share and reuse the same information

Standard model and query language

- The relational model and SQL are very well-known standards
- The same theoretical background is shared by the different implementations
- This ensures application portability

Robustness

- Have been used for over 40 years

Weaknesses of RDBMS

Impedance mismatch

- Data are stored according to the relational model, but applications to modify them typically rely on the object-oriented model: objects or class definitions must be mapped to database tables defined by a relational schema but the two models has different expressivity, different constraints, ecc..
- Many solutions, no standard
 - E.g.: Object Oriented DBMS (OODBMS), Object-Relational DBMS (ORDBMS), Object-Relational Mapping (ORM) frameworks

Painful scaling-out

- Not suited for a cluster architecture
- Distributing an RDBMS is far from being easy

Consistency vs latency

- Consistency is a must – even at the expense of latency
- Today's applications require high reading/writing throughput with low latency

Schema rigidity

- Schema evolution is often expensive

What we need to know

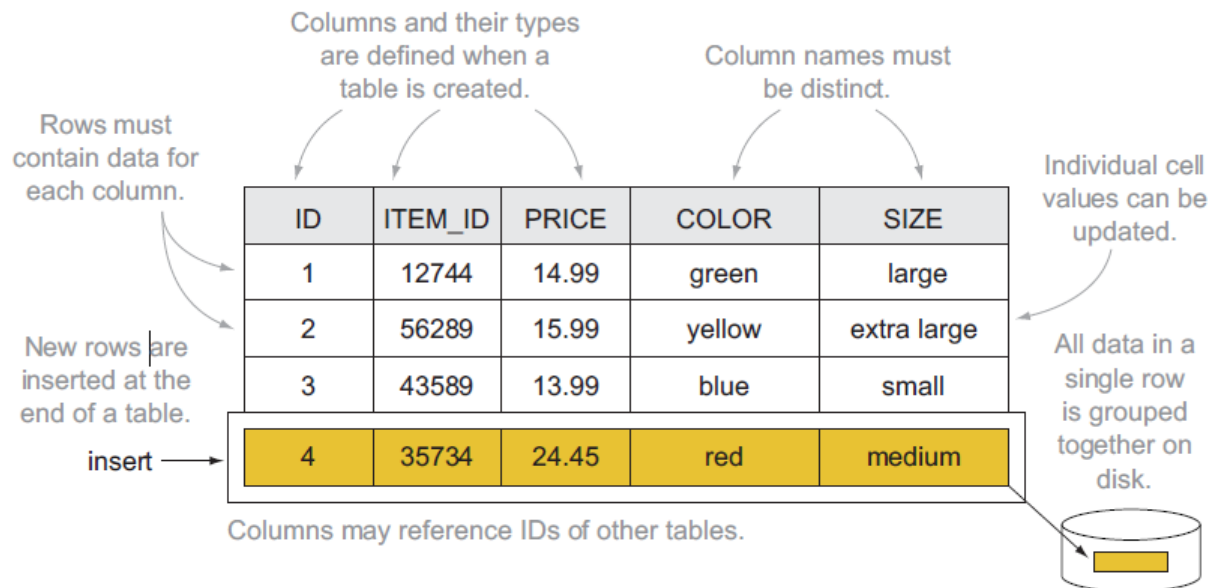
To properly work with NoSQL we need to know

- How to choose a data model and its consequences
- How to model aggregates
- How to handle data and on a cluster
 - Sharding
 - Replication
 - Consistency – Availability models

Data models

Relational model

Based on tables and rows

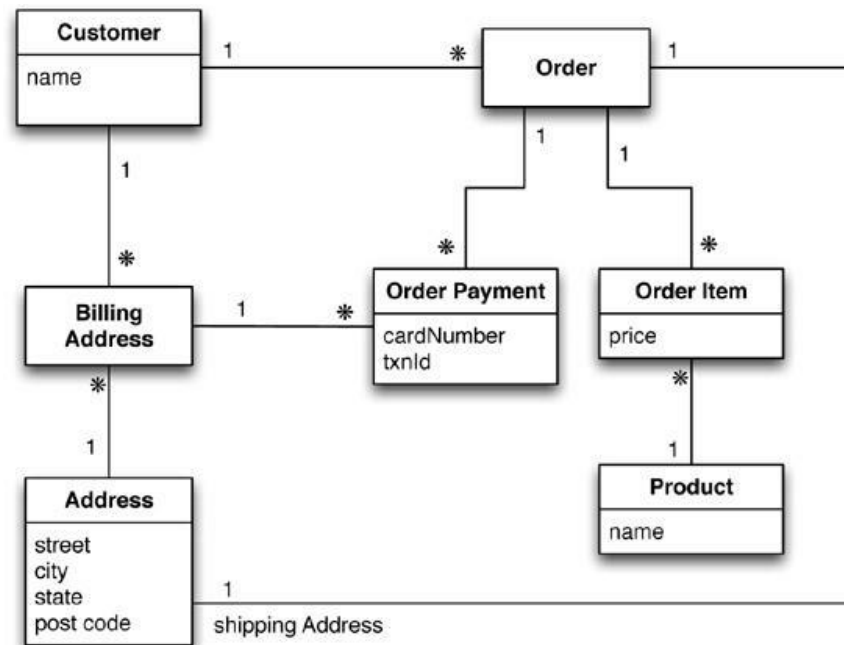


NoSQL: several data models

One of the key challenges is to understand which one fits best with the required application

Model	Description	Use cases
Key-value	Associates any kind of value to a string	Dictionary, lookup table, cache, file and images storage
Document	Stores hierarchical data in a tree-like structure	Documents, anything that fits into a hierarchical structure
Wide column	Stores sparse matrixes where a cell is identified by the row and column keys	Crawling, high-variability systems, sparse matrixes
Graph	Stores vertices and arches	Social network queries, inference, pattern matching

An ER running example



Relational modeling

Customer	
Id	Name
1	Martin

Orders		
Id	CustomerId	ShippingAddressId
99	1	77

Product	
Id	Name
27	NoSQL Distilled

BillingAddress		
Id	CustomerId	AddressId
55	1	77

OrderItem			
Id	OrderId	ProductId	Price
100	99	27	32.45

Address	
Id	City
77	Chicago

OrderPayment				
Id	OrderId	CardNumber	BillingAddressId	txnId
33	99	1000-1000	55	abelif879rft

Key-value: data model

Each DB contains one or more **collections** (corresponding to tables)

Each collection contains a list of **key-value pairs**

- Key: a unique string
 - E.g.: ids, hashes, paths, queries, REST calls
- Value: a BLOB (binary large object)
 - E.g.: text, documents, web pages, multimedia files

Atomicity level: the key-value pair

Sharding: based on keys

Looks like a simple dictionary

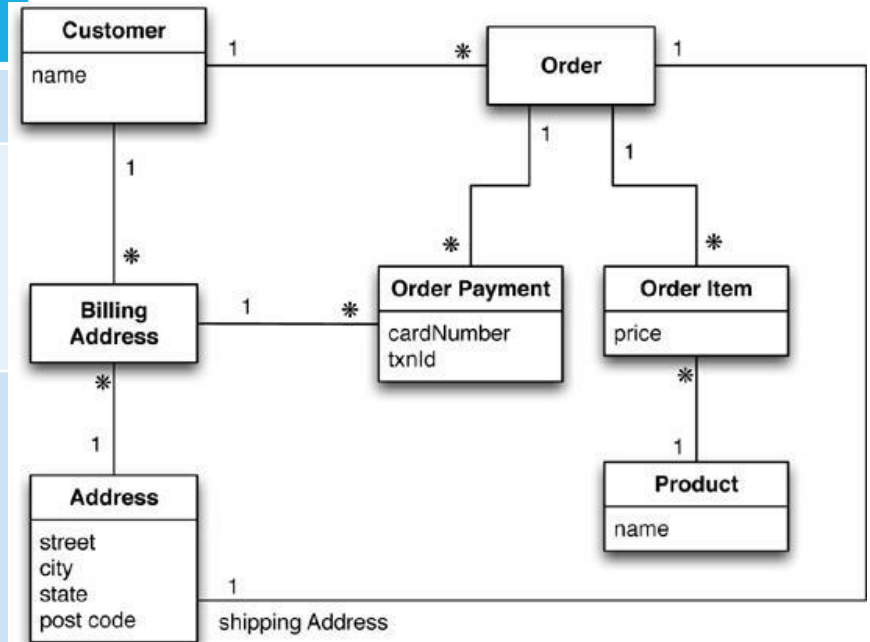
- The collection is indexed by key
- The value may contain several information: one or more definitions, synonyms and antonyms, images, etc.
- Rows can have completely different schemas

Key	Value
image-12345.jpg	Binary image file
http://www.example.com/my-web-page.html	HTML of a web page
N:/folder/subfolder/myfile.pdf	PDF document
9e107d9d372bb6826bd81d3542a419d6	The quick brown fox jumps over the lazy dog
view-person?person-id=12345&format=xml	<Person><id>12345</id>.</Person>
SELECT PERSON FROM PEOPLE WHERE PID="12345"	<Person><id>12345</id>.</Person>

Amazon Dynamo DB adopts the key-value model

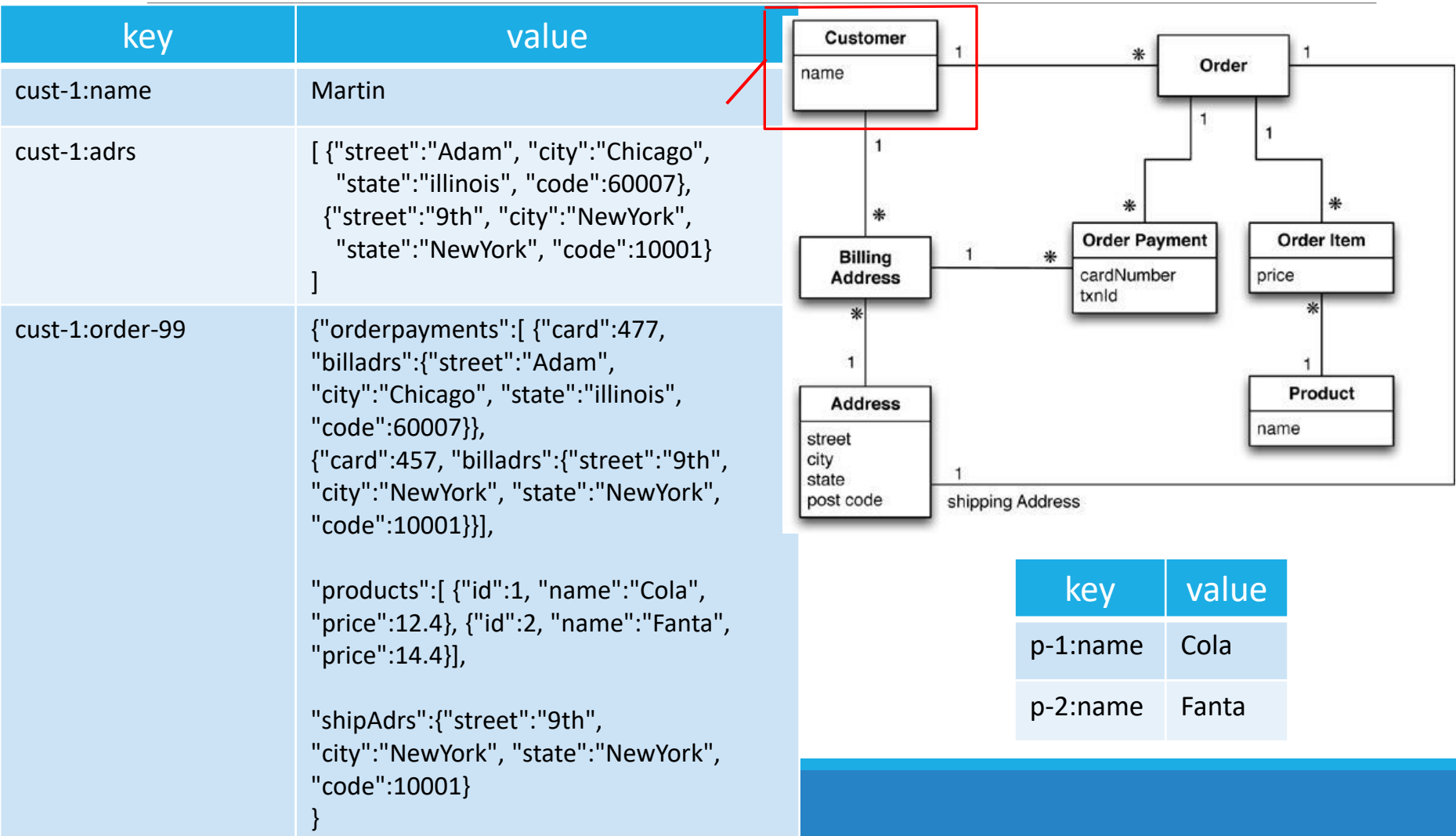
Key-value modeling

key	value
cust-1:name	Martin
cust-1:adrs	[{"street":"Adam", "city":"Chicago", "state":"illinois", "code":60007}, {"street":"9th", "city":"NewYork", "state":"NewYork", "code":10001}]
cust-1:order-99	{"orderpayments":[{"card":477, "billadrs":{"street":"Adam", "city":"Chicago", "state":"illinois", "code":60007}}, {"card":457, "billadrs":{"street":"9th", "city":"NewYork", "state":"NewYork", "code":10001}}}, {"products":[{"id":1, "name":"Cola", "price":12.4}, {"id":2, "name":"Fanta", "price":14.4}], {"shipAdrs":{"street":"9th", "city":"NewYork", "state":"NewYork", "code":10001}}



key	value
p-1:name	Cola
p-2:name	Fanta

Key-value modeling



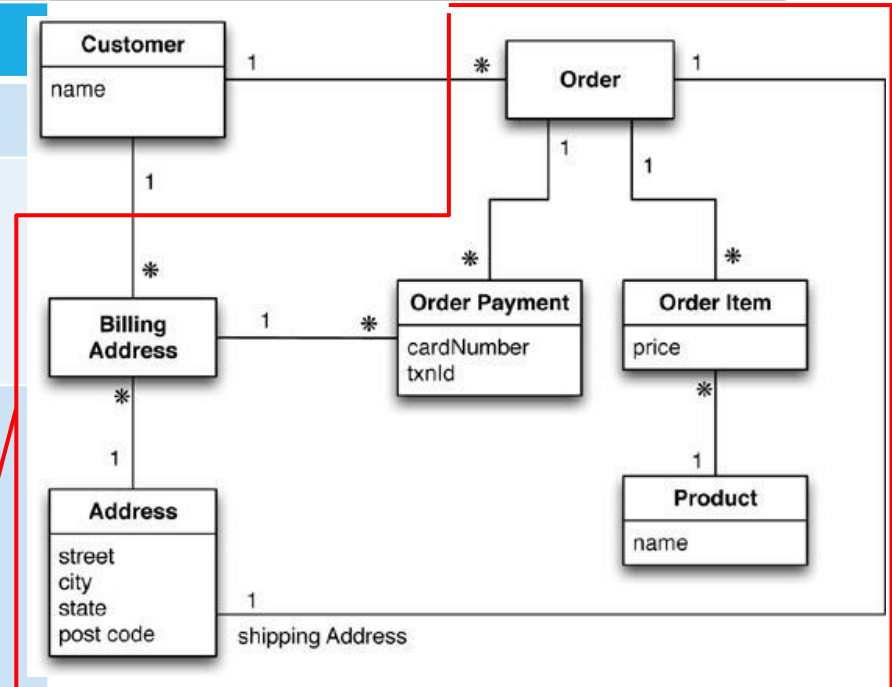
Key-value modeling

key	value	<pre> erDiagram Customer --o{ Billing-Address : "has" Billing-Address --o{ Address : "has" Address --o{ Address : "has" Customer --o{ Order : "has" Order --o{ Order-Payment : "has" Order --o{ Order-Item : "has" Order-Item --o{ Product : "has" Billing-Address --o{ Order-Payment : "has" </pre>	
cust-1:name	Martin		
cust-1:adrs	[{"street":"Adam", "city":"Chicago", "state":"illinois", "code":60007}, {"street":"9th", "city":"NewYork", "state":"NewYork", "code":10001}]		
cust-1:order-99	{ "orderpayments": [{"card":477, "billadrs":{"street":"Adam", "city":"Chicago", "state":"illinois", "code":60007}}, {"card":457, "billadrs":{"street":"9th", "city":"NewYork", "state":"NewYork", "code":10001}}], "products": [{"id":1, "name":"Cola", "price":12.4}, {"id":2, "name":"Fanta", "price":14.4}], "shipAdrs":{"street":"9th", "city":"NewYork", "state":"NewYork", "code":10001} }		

key	value
p-1:name	Cola
p-2:name	Fanta

Key-value modeling

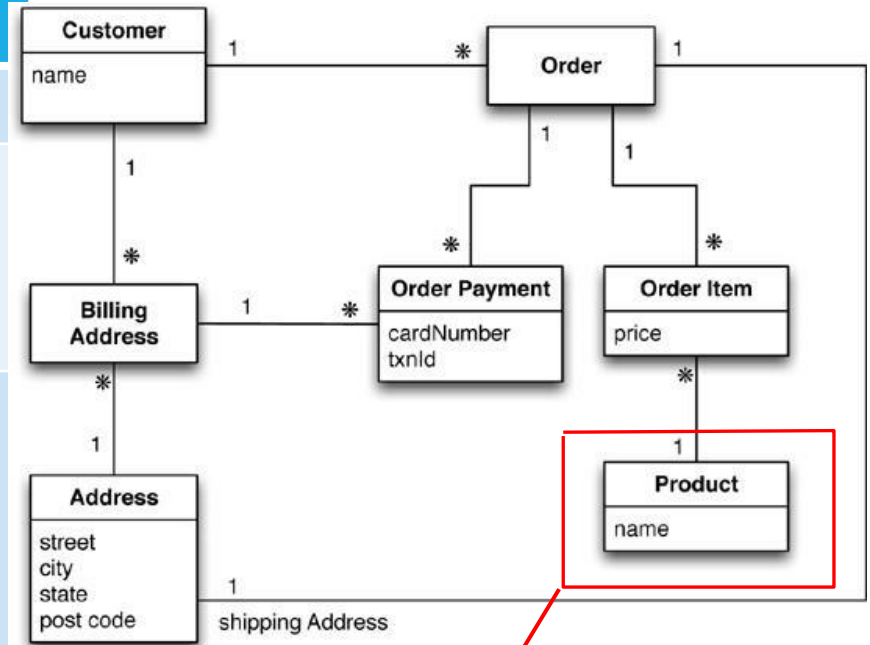
key	value
cust-1:name	Martin
cust-1:adrs	[{"street":"Adam", "city":"Chicago", "state":"illinois", "code":60007}, {"street":"9th", "city":"NewYork", "state":"NewYork", "code":10001}]
cust-1:order-99	<pre>{"orderpayments":[{"card":477, "billadrs":{"street":"Adam", "city":"Chicago", "state":"illinois", "code":60007}}, {"card":457, "billadrs":{"street":"9th", "city":"NewYork", "state":"NewYork", "code":10001}}], "products":[{"id":1, "name":"Cola", "price":12.4}, {"id":2, "name":"Fanta", "price":14.4}], "shipAdrs":{"street":"9th", "city":"NewYork", "state":"NewYork", "code":10001}}</pre>



key	value
p-1:name	Cola
p-2:name	Fanta

Key-value modeling

key	value
cust-1:name	Martin
cust-1:adrs	[{"street":"Adam", "city":"Chicago", "state":"illinois", "code":60007}, {"street":"9th", "city":"NewYork", "state":"NewYork", "code":10001}]
cust-1:order-99	<pre>{"orderpayments": [{"card":477, "billadrs":{"street":"Adam", "city":"Chicago", "state":"illinois", "code":60007}}, {"card":457, "billadrs":{"street":"9th", "city":"NewYork", "state":"NewYork", "code":10001}}], "products": [{"id":1, "name":"Cola", "price":12.4}, {"id":2, "name":"Fanta", "price":14.4}], "shipAdrs":{"street":"9th", "city":"NewYork", "state":"NewYork", "code":10001} }</pre>



key	value
p-1:name	Cola
p-2:name	Fanta

Key-value: querying

Three simple kinds of query:

- `put($key as xs:string, $value as item())`
 - Adds a key-value pair to the collection
 - If the key already exists, the value is replaced
- `get($key as xs:string) as item()`
 - Returns the value corresponding to the key (if it exists)
- `delete($key as xs:string)`
 - Deletes the key-value pair

The value is a *black box*: it cannot be queried!

- No "where" clauses
- No indexes on the values
- Schema information is often indicated in the key

Key	Value
user:1234:name	Enrico
user:1234:age	30
post:9876:written-by	user:1234
post:9876:title	NoSQL Databases
comment:5050:reply-to	post:9876

Document: data model

Each DB contains one or more **collections** (corresponding to tables)

Each collection contains a list of **documents** (usually JSON)

- Documents are hierarchically structured

Each document contains a set of **fields**

- The **ID** is mandatory

Each field corresponds to a **key-value pair**

- Key: unique string in the document
- Value: either simple (string, number, boolean) or complex (object, array, BLOB)
 - A complex field can contain other field

Atomicity level: the document

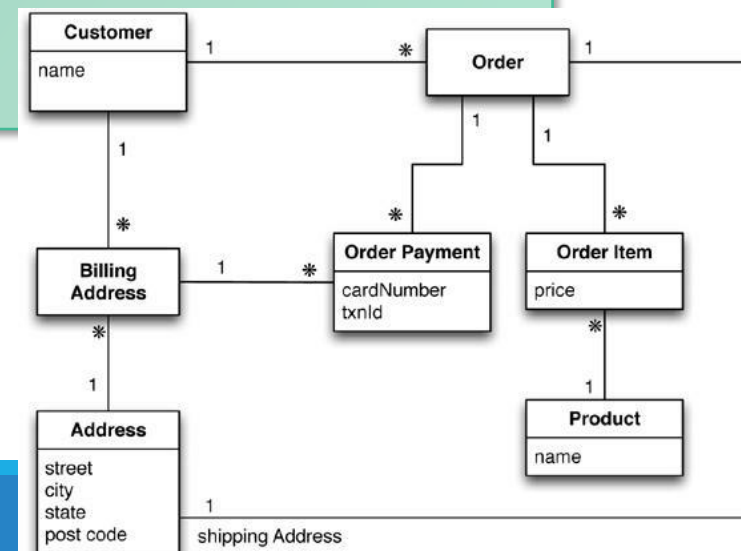
MongoDB adopts the document model

```
{
  "_id": 1234,
  "name": "Enrico",
  "age": 30,
  "address": {
    "city": "Ravenna",
    "postalCode": 48124
  },
  "contacts": [ {
    "type": "office",
    "contact": "0547-338835"
  }, {
    "type": "skype",
    "contact": "egallinucci"
  } ]
}
```

Document modeling 1

```
{  "_id": 1,
  "name": "Martin",
  "adrs": [ {"street":"Adam", "city":"Chicago", "state":"illinois", "code":60007},
            {"street":"9th", "city":"NewYork", "state":"NewYork", "code":10001}
        ],
  "orders": [
    {"orderpayments": [{"card":477, "billadrs":{"street":"Adam", "city":"Chicago", "state":"illinois", "code":60007}},
                      {"card":457, "billadrs":{"street":"9th", "city":"NewYork", "state":"NewYork", "code":10001}}
    ],
    "products": [{"id":1, "name":"Cola", "price":12.4},
                 {"id":2, "name":"Fanta", "price":14.4}
    ],
    "shipAdrs":{"street":"9th", "city":"NewYork", "state":"NewYork", "code":10001}
  }
}
```

```
[
  {
    "_id":1,
    "name":"Cola",
    "price":12.4
  },
  {
    "_id":2,
    "name":"Fanta",
    "price":14.4
  }
]
```

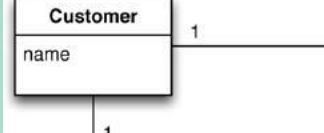


Document modeling 2

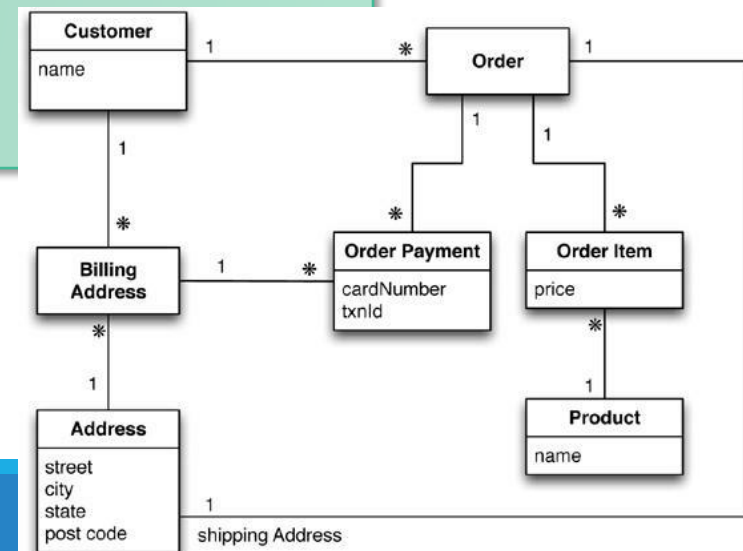
```
{  "_id": 1,
   "name": "Martin",
   "adrs": [ {"street": "Adam", "city": "Chicago", "state": "illinois", "code": 60007},
             {"street": "9th", "city": "NewYork", "state": "NewYork", "code": 10001}
 ]
}
```

Requires a join to put together information about customers and products

```
{ "_id": 1,
  "customer": 1,
  "orderpayments": [ { "card": 477, "billadr": { "street": "Adam", "city": "Chicago", "state": "illinois", "code": 60007 },
                      { "card": 457, "billadr": { "street": "9th", "city": "NewYork", "state": "NewYork", "code": 10001 } }
                    ],
  "products": [ { "id": 1, "name": "Cola", "price": 12.4 },
                 { "id": 2, "name": "Fanta", "price": 14.4 }
                ],
  "shipAdrs": { "street": "9th", "city": "NewYork", "state": "NewYork", "code": 10001 }
}
```



```
{  "_id":1,
    "name":"Cola",
    "price":12.4
},
{"_id":2,
  "name":"Fanta",
  "price":14.4
}
```



Document modeling 3

```
{ "_id": 1,
  "customer": { "id": 1, "name": "Martin", "adrs": [ { "street": "Adam", "city": "Chicago", "state": "illinois", "code": 60007 },
                                                    { "street": "9th", "city": "NewYork", "state": "NewYork", "code": 10001 }
          ]
},
  "orderpayments": [ { "card": 477, "billadrs": { "street": "Adam", "city": "Chicago", "state": "illinois", "code": 60007 },
                     { "card": 457, "billadrs": { "street": "9th", "city": "NewYork", "state": "NewYork", "code": 10001 } }
],
  "products": [ { "id": 1, "name": "Cola", "price": 12.4 },
                { "id": 2, "name": "Fanta", "price": 14.4 }
],
  "shipAdrs": { "street": "9th", "city": "NewYork", "state": "NewYork", "code": 10001 }
}
```

```
{  "_id": 1,
    "name": "Cola",
    "price": 12.4
},
{  "_id": 2,
    "name": "Fanta",
    "price": 14.4
}
```

```
{  "_id": 1,
    "name": "Martin",
    "adrs": [ { "street": "Adam", "city": "Chicago", "state": "illinois", "code": 60007 },
              { "street": "9th", "city": "NewYork", "state": "NewYork", "code": 10001 }
            ]
}
```

Customers are
redundantly stored
in each order

Document: querying

Differently from the key-value, the value is *visible* by the DBMS

Thus, query languages are quite expressive

- Can create indexes on fields
- Can filter on the fields
- Can return more documents with one query
- Can select which fields to project
- Can update specific fields

Different implementations, different functionalities

- Some enable (possibly materialized) views
- Some enable MapReduce queries
- Some provide connectors to Big Data tools (e.g., Spark, Hive)
- Some provide *full-text search* capabilities

Wide column: data model

Each DB contains one or more **sparsely populated tables whose rows can contain arbitrary columns**.

The wide column model can be seen as a two-level aggregate structure

- As with key-value stores, the first key is often described as a row identifier, picking up the aggregate of interest
- This row aggregate is itself formed of a map of more detailed values. These second-level values are referred to as columns, each being a key-value pair
- Each column is a key-value pair itself

Columns are organized into column families. Each column has to be part of a single column family (data for a particular column family will be usually accessed together) **column families** (corresponding to relational tables)

Wide-column modeling

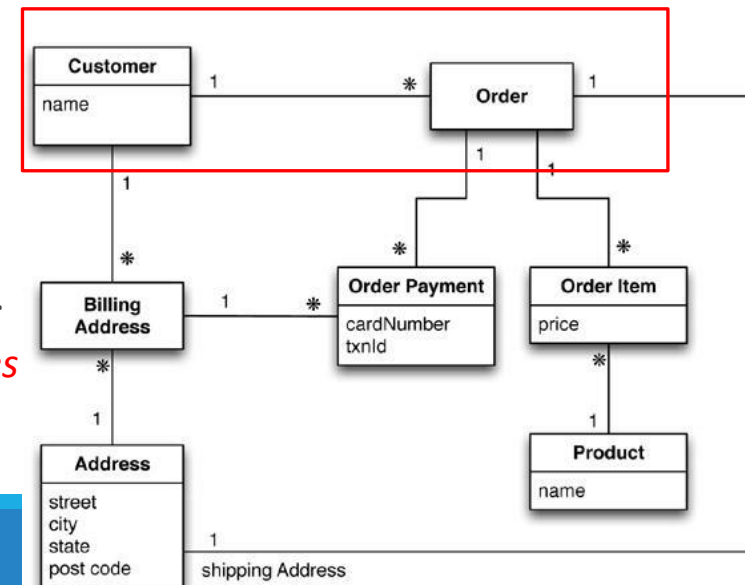
Key	CName	BillingAddr	...	Order1Date	Order1CardN		Order2Date	Order2CardN	
1	Martin	TrafalgarSqr	...	18/10/23	1237 2558 5566		23/05/2023	2558 77556 7566	
2									

Key Column Family 1 Column Family 2

Two ways to think about how the data is structured:

- **Row-oriented:** Each row is an aggregate (e.g., customer with the ID of 1) with column families representing useful chunks of data (e.g., customer profile, order) within that aggregate
- **Column-oriented:** Each column family defines a record type (e.g., customer profiles) with rows for each of the records. You then think of a row as the join of records in all column families.

Column Families can be then, to some extent, considered as tables in RDBMSs. Unlike table in RDBMSs, a Column Family can have different columns for each row it contains



Wide-column modeling

Key	CName	BillingAddr	...	Order1Date	Order1CardN		Order2Date	Order2CardN	
1	Martin	TrafalgarSqr	...	18/10/23	1237 2558 5566		23/05/2023	2558 77556 7566	
2									

Key Column Family 1 Column Family 2

Two ways to think about

- **Row-oriented:** Each row is a record (e.g., the ID of 1) with columns of data (e.g., customer name, address, etc.)
- **Column-oriented:** Each column is a record (e.g., customer promises), with rows for each of the records. You

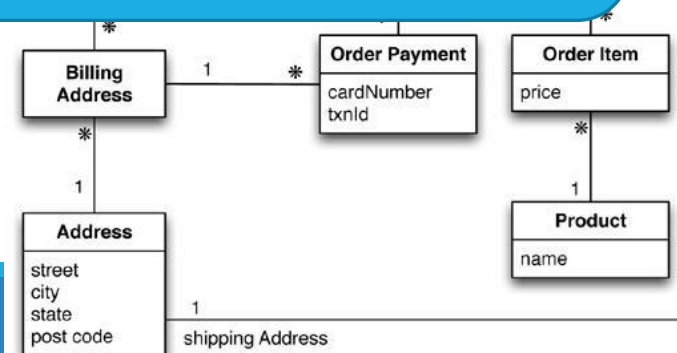
Each column is actually stored as key-value pair

BillingAddr:TrafalgarSquare

Only valued columns are stored for each row: the matrix is sparse

(e.g., customer promises), with rows for each of the records. You then think of a row as the join of records in all column families.

Column Families can be then to some extent considered as tables in RDBMSs. Unlike table in RDBMSs, a Column Family can have different columns for each row it contains



Wide column: data model

Atomicity level: the row

Sharding:

- based on a set of attributes
 - Sharding on the rowkey it is not recommended since a selection access would impact on several shards
- it is applied on a row

With respect to the relational model:

- Rows specify only the columns for which a value exists
 - Particularly suited for sparse matrixes
- Timestamps can be used to defines *versions* of column values

Google Big table and Hbase adopts the wide-column model

Wide column: querying

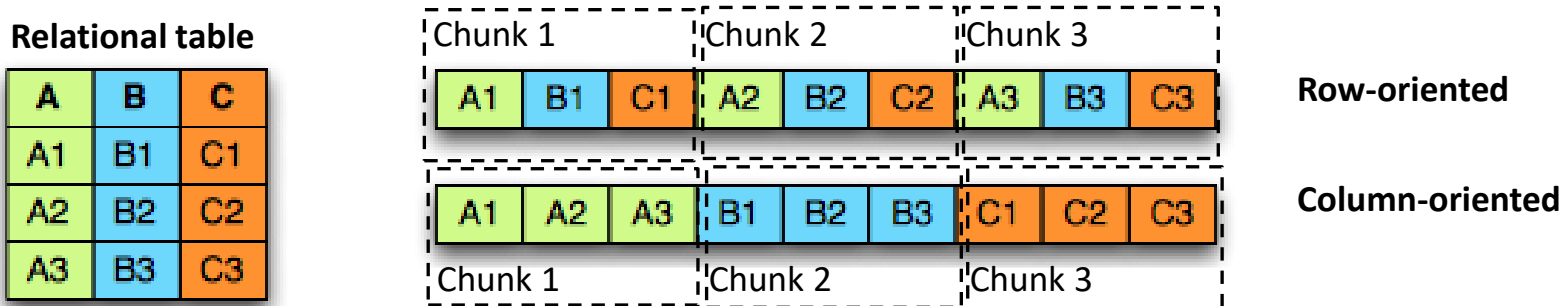
The query language expressiveness is in between key-value and document data models

- Column indexes are discouraged
- Can filter on column values (not always)
- Can return more rows with one query
- Can select which columns to project
- Can update specific columns (not always)

Given the similarity with the relational model, a **SQL-like** language is often used

Wide column: $\neq$ columnar

Do not mistake the wide column data model with the columnar storage where tables are stored in physical chunks as sequences column values, rather than sequences of row values. Columnar DBs has fixed schemas. Columnar DBs are the best for applications that manage large amounts of data and require many reads and few updates (e.g. Data Warehouses).



Row-oriented

- ◆ **Pros:** inserting a record is easy
- ◆ **Cons:** several unnecessary data may be accessed when reading a record

Column-oriented

- ◆ **Pros:** only the required values are accessed
- ◆ **Cons:** writing a record requires multiple accesses

Graph: data model

Each DB contains one or more **graphs**

Each graph contains **vertices** and **arcs**

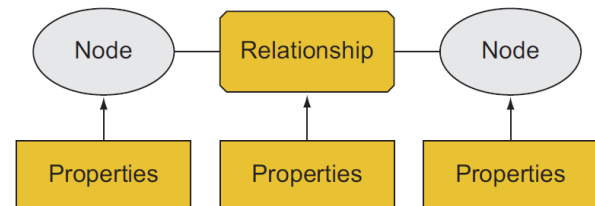
- Vertices: usually represent real-world entities
 - E.g.: people, organizations, web pages, workstations, cells, books, etc.
- Arcs: represent directed relationships between the vertices
 - E.g.: friendship, work relationship, hyperlink, ethernet links, copyright, etc.
- Vertices and arcs are described by **properties**

Atomicity level: the transaction

Most known specializations:

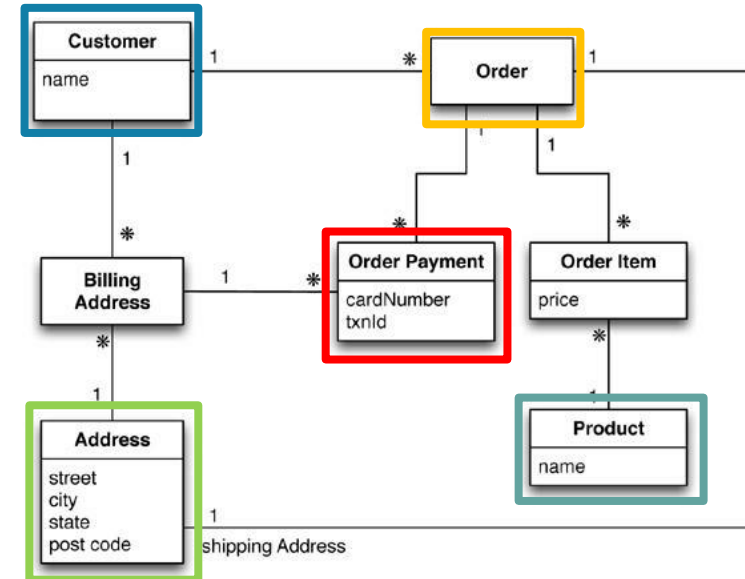
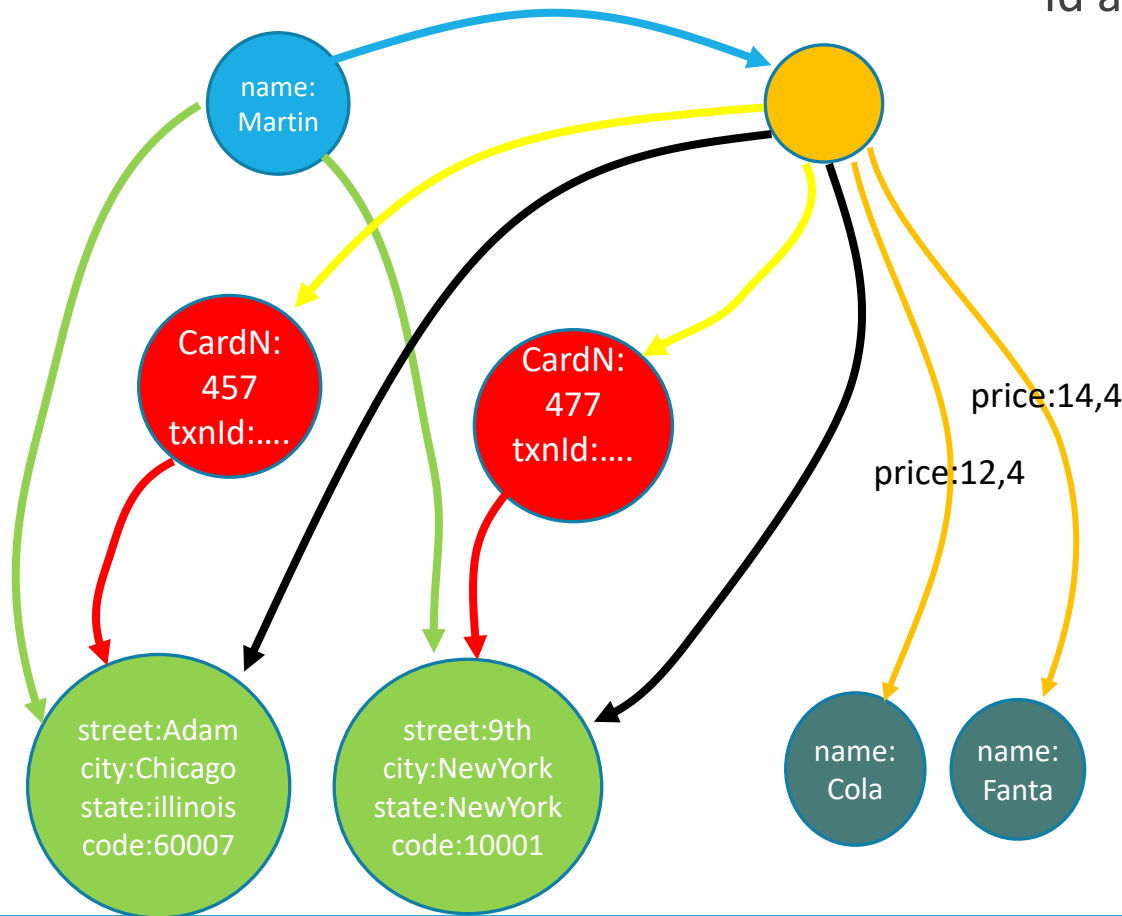
- Reticular data model
 - Parent-child or owner-member relationships
- Triplestore
 - Subject-predicate-object relationships (e.g., RDF)

Neo4J adopts the graph data model



Graph modeling

Id are implicitly handled by the DBMS



Graph: querying

Graph databases usually model completely different contexts

Thus, query language and mechanism is quite different

- Support for transactions
- Support for indexes, selections and projections
- Query language is based on patterns

Query	Pattern
Find friends of friends	(user)-[:KNOWS]-(friend)-[:KNOWS]-(foaf)
Find shortest path from A to B	shortestPath((userA)-[:KNOWS*..5]-(userB))
What has been bought by those who bought my same products?	(us1)-[:PURCHASED]->(prod)<-[:PURCHASED]-(us2)-[:PURCHASED]->(otherProd)

In the Neo4J syntax:

- () identifies nodes (*name*) *name* is a label for a generic node
- [] identifies arcs
- - - / - -> identifies a relationships independently/dependently on its direction
- :TYPE constraints on a node/arc type

Graph vs Aggregate-oriented

The graph data model is intrinsically different from the others

- Focused on the relationships rather than on the entities per-se
- Limited scalability
- Data-driven modeling (more focus on atomic data like relational model)

Other data models (key-value, document and wide column) are defined **aggregate-oriented**

- The aggregate is the atomic block
- More encapsulation, less joins
- High scalability
- Query-driven modeling

Why scalability is limited in Graph DBMS?

Real graphs are strongly connected:

- **Six degrees of separation** is the **theory** that any person on the planet can be connected to any other person on the planet through a chain of acquaintances that has no more than five intermediaries

This makes very hard, or even impossible, to shard a graph on several machines without "*cutting*" several arcs (i.e. having several cross-machine links)

Applications will mostly probably query across machine boundaries due to data normalization and absence of redundancy that prevent joins in aggregate modeling

Different approaches can be used to limit the effects:

- Batching cross machine queries, so we only perform them at the close of each breadth first step.
- Limiting the depth of cross machine node searches (FlockDB is optimized for being very fast in returning one-hop relationships)

One size does not fit all

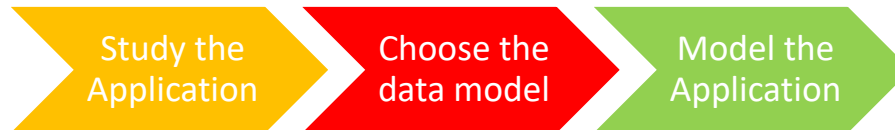
TO EACH APPLICATION ITS OWN DATA MODEL

One size does not fit all!

- ❑ For more than 40 years the only available data model has been the relational one
 - ✓ Applications with different needs/features have been implemented on the same model
 - ✓ Performance was limited



- ❑ The need to support applications with strict performance constraints and huge data sets led to the emergence of NoSQL DBMSs
 - ✓ Each model has different features, specific to workloads
 - ✓ The designer must know the main features of the different data models



One size does not fit all!

- ❑ For more than 40 years the only available data model has been the relational one
 - ✓ Applications with different needs/features have been implemented on the same model
 - ✓ Performance was limited

Modello	Descrizione	Casi d'uso	Applicazioni
Key-value	Associates any kind of value to a string	Dictionary, lookup table, cache, file and images storage	Web session profile, shopping cart, user preferences
Document	Stores hierarchical data in a tree-like structure	Documents, anything that fits into a hierarchical structure	Event log, CMS, blogging platform
Wide column	Stores sparse matrixes where a cell is identified by the row and column keys	Crawling, high-variability systems, sparse matrixes	Event log, CMS, blogging platform, GIS
Graph	Stores vertices and arches	Social network queries, inference, pattern matching	Social network, routing application, fraud detection

s and huge

Study the
Application

Choose the
data model

Model the
Application

One size does not fit all!

- ❑ For more than 40 years the only available data model has been the relational one
 - ✓ Applications with different needs/features have been implemented on the same model
 - ✓ Performance was limited

Modello	Descrizione	Casi d'uso	Applicazioni
Key-value	Associates any kind of value to a string	Dictionary, lookup table, cache, file and images storage	Web session profile, shopping cart, user preferences
Document	Stores hierarchical data in a tree-like structure	Documents, anything that fits into a hierarchical structure	Event log, CMS, blogging platform
Wide column	Stores sparse matrixes where a cell is identified by the row and column keys	Crawling, high-variability systems, sparse matrixes	Event log, CMS, blogging platform, GIS
Graph	Stores vertices and arches	Social network queries, inference, pattern matching	Social network, routing application, fraud detection

Aggregate
Modelling

and huge

Study the
Application

Choose the
data model

Model the
Application

Popular NoSQL DBMSs

Provider / Data Model	Key-Value	Document	Wide Column	Graph-Based
AWS	DynamoDB, ElastiCache (=Redis), S3	DocumentDB	Keyspaces (=Cassandra), SimpleDB	Neptune
Google	MemoryStore	Firestore	BigTable	AuraDS (=Neo4J)
Azure	Cosmos DB	Cosmos DB	Cosmos DB (include Azure Table Storage)	Cosmos DB
Apache	Ignite	CouchDB	HBase	Giraph, TinkerPop
Other	Redis, Memcached	MongoDB	Cassandra, Scylla DB	Neo4J, TigerGraph, JanusGraph

Key-Value DBMSes

When to use

- **Store session informations**
 - Each web session is identified by its own sessionId. All related data can be stored with a PUT request and returned with a GET request.
- **User profiles, preferences**
 - Each user is uniquely identified (userId, username) and has her own preferences in terms of language, colors, time zone, products, etc. – data that fits well within an aggregate
- **Shopping cart, chat services**
 - Each e-commerce websites associates a shopping cart to a user; it can be stored as an aggregate identified by the user ID.

Document DBMS

When to use

- **Event logs**
 - Often used as [central repositories to store event logs from several applications](#)
 - It is a good practice to shard on the app name or on the type of event
- **CMS, blogging platforms**
 - [The absence of a predefined schema fits well](#) within content management systems (CMS) or website management applications, to handle comments, registrations and user profiles
- **Web Analytics or Real-Time Analytics**
 - [The ability to update only specific fields](#) enables fast update of analytical metrics
 - [Text indexing](#) enables real-time sentiment analysis and social media monitoring
- **E-commerce applications**
 - [Schema flexibility is often required](#) to store products and orders, as well as to enable schema evolution without incurring into refactoring or migration costs

Graph DBMS

When to use:

- **Interlinked data**
 - [Social networks](#) are one of the most typical use case of graph databases (e.g., to store friendships or work relationships); [every relationship-centric domain is a good one](#)
- **Routing and location-based services**
 - Applications working on the [TSP \(Travelling Salesman Problem\)](#) problem
 - Location-based application that, for instance, recommend the best restaurant nearby; in this case, [relationships model the distance between node](#)
- **Recommendation applications, fraud-detection**
 - Systems recommending «the products bought by your friends», or «the products bought by those who bought your same products»
 - When relationships model behaviors, outlier detection may be useful to identify frauds

Polyglot persistence

Different databases are designed to solve different problems

Using a single DBMS to handle everything ...

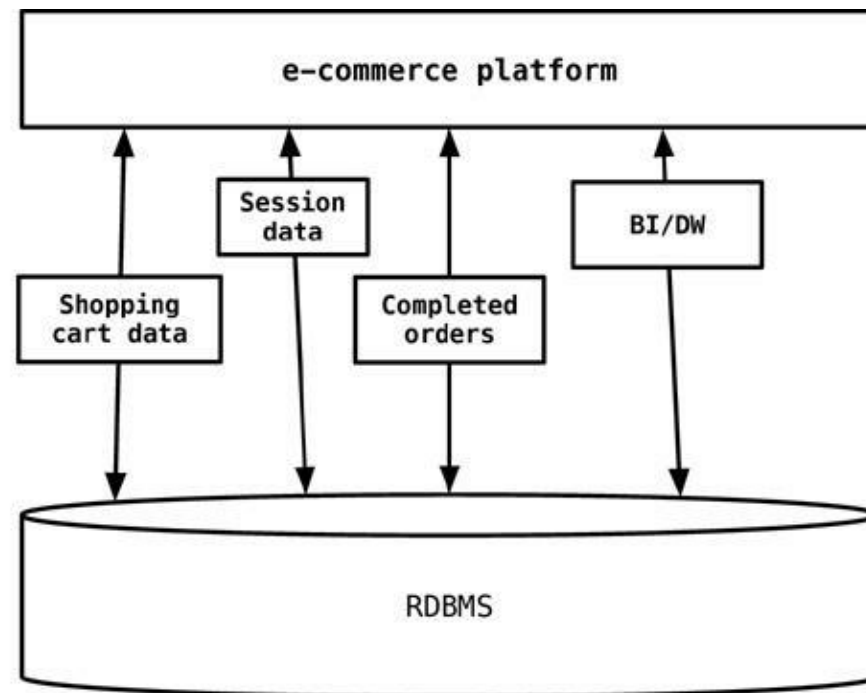
- Operational data
- Temporary session information
- Graph traversing
- OLAP analyses
- ...



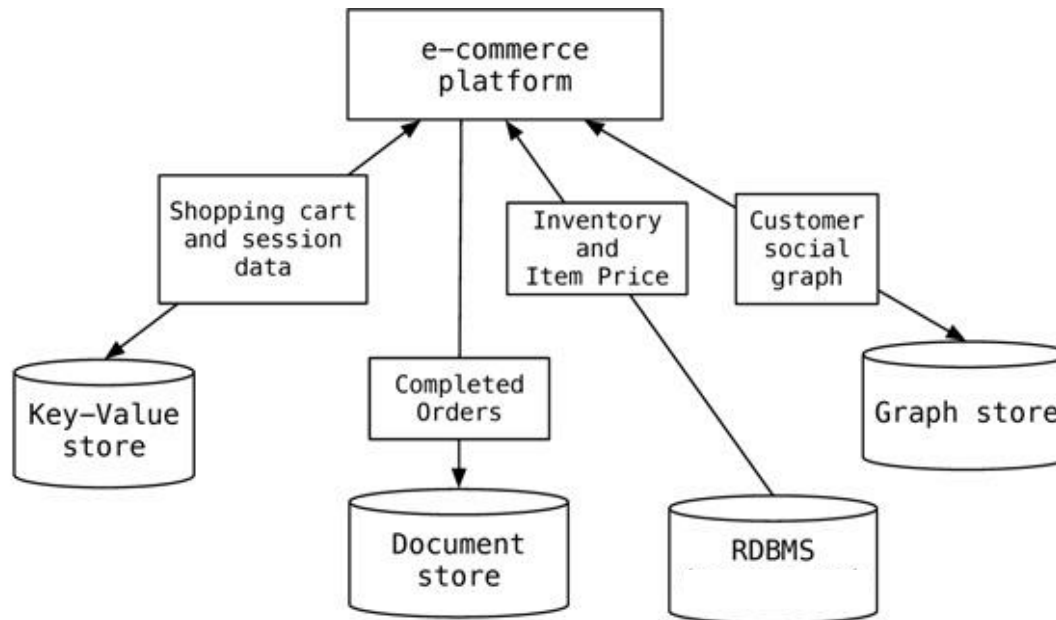
... usually lead to inefficient solutions

Each activity has its own requirements (availability, consistency, fault tolerance, etc.)

Traditional approach



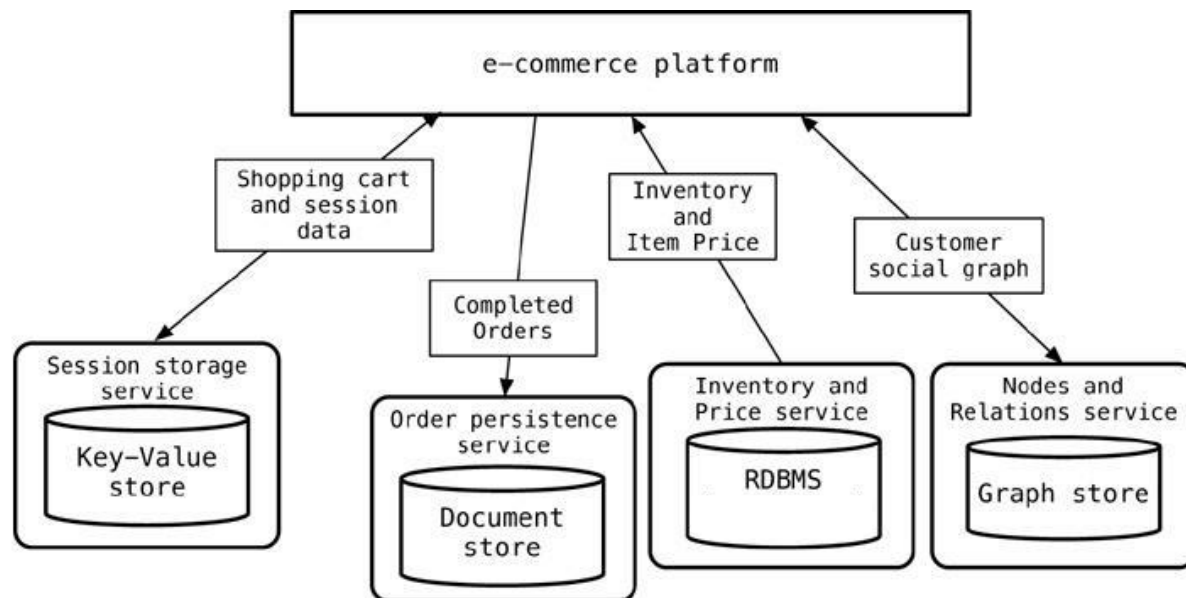
Polyglot data management



Service-oriented polyglot data management

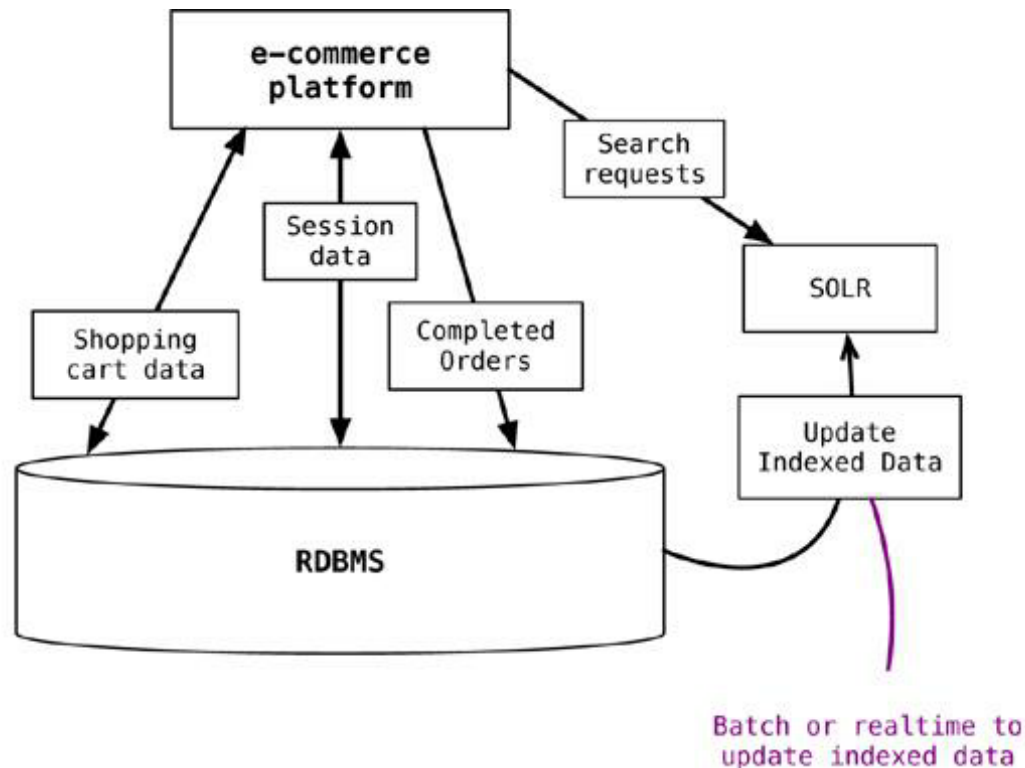
Each DB should be "embedded" within services, which offer API services to enable data access and manipulation

- Several NoSQL systems (e.g., Riak, Neo4J) already provide REST APIs



Supporting existing technologies

If the current solution cannot be changed, NoSQL systems can still support the existing ones



Polyglot persistence in the business

DBAs are becoming more and more polyglot

- Important to understand how these systems work and how they can be exploited

Beware about security

- Many NoSQL systems are still young and **do not** offer strong security mechanisms
- Responsibility is dumped on the applications

What about licences, support, upgrade, drivers, auditing, support systems

- Most systems are open-source and have an active community; some companies offer enterprise support
- Support tool are being released (e.g., MongoDB Monitoring Service, Datastax Ops Center, Rekon browser)
- ETL tools are adding support for NoSQL systems

Distribution Models

A LOOK BEHIND THE CURTAIN

Distribution Models

One of the strengths of NoSQL systems is their **scale-out capability**

- The aggregate is well suited for being distributed within a cluster
- Graph databases do not scale as well as the others
- NoSQL systems can be used in a single server environment too

Two aspects must be considered when deploying on a cluster

- **Sharding:** distributing the data across different nodes
- **Replication:** creating copies of the data on several nodes
 - Master-slave
 - Peer-to-peer

Sharding & Replication Goals

Robustness

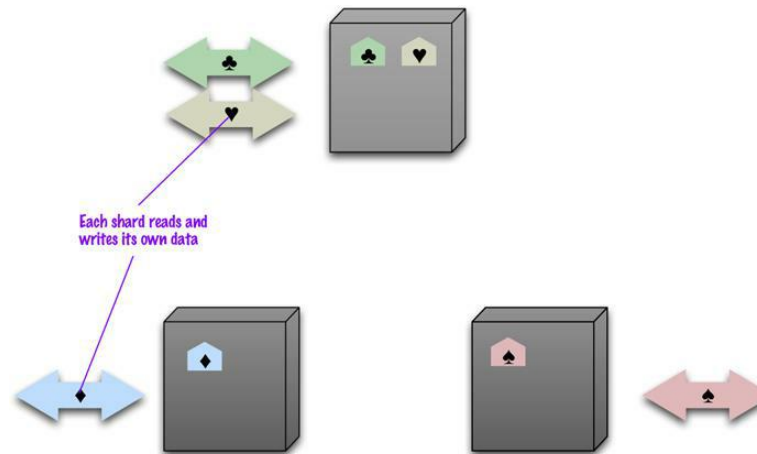
- **Replication**: in case of node failure, replicas prevent data loss

Efficiency

- **Replication**: improves efficiency since data can be read in parallel by several users on distinct copies. Furthermore, reads has more chances to happen locally to the node in charge to handle the replicas. If only one copy exists, it will have higher probability to be *far* from the node in charges to handle it, thus overall the system will occur higher network costs.
- **Sharding**: improves efficiency since different data partitions can be handled in parallel by different nodes

Sharding

Sharding: subdividing the data in *shards* that are stored in different machines



A good *sharding strategy* is **fundamental** to optimize performances

- Usually based on one or more fields composing the sharding key

Sharding strategy

Thumbs-up rules for a sharding strategy:

1. Data-locality: stores the data close to those that need to access them
 - E.g., store orders of Italian customers in the European data center
2. Keep a balanced distribution
 - Each node should have the same percentage of data (more or less)
3. Respect the domain optimization goal

Optimization Goals

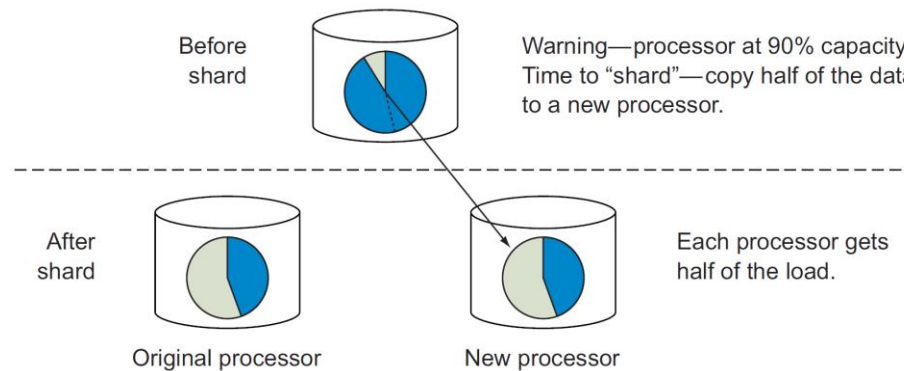
Depending on the workload features two optimization goals can be pursued

1. **Inter-query balancing:** Useful when the workload is composed by several light queries (i.e. queries accessing few data)
 - Store data to be accessed by each query together in order to avoid network costs
 - Balance the workload by distributing the workload, that is: different queries will access different nodes
 - **This is the case for operational workload (e.g. shopping carts, user profiles)**
2. **Intra-query balancing:** Useful when the workload is composed by few heavy queries (i.e. queries accessing tons of data)
 - Store in separate nodes data accessed by a single query in order to exploit several nodes to handle it
 - Balance the workload by distributing each single query
 - **This is the case for analytic workload (e.g. OLAP queries)**

Auto-sharding

Many NoSQL systems offer **auto-sharding policies**

- The database distributes the data according to the workload



Beware: redefining (or choosing later) the sharding strategy can be quite expensive

Sharding strategy

Sharding is carried through an attribute of the record called **sharding key**

There are some typical strategies for sharding:

- **Range strategy:** each partition contains a range of data **sorted** on the basis of a key;
 - HBASE and Google spanner adopts this strategy
 - **Pros:** efficiently run range queries that works on the sharding key values. All the rows are stored at the same node
 - **Cons:** some nodes can become bottlenecks if workload focuses on a specific range of values
 - **Cons:** ranges are defined a priori and this can determine heavy data redistribution
- **Hash strategy:** a **hash function** is used to allocate data to partitions
 - DynamoDB and Cassandra adopts this strategy
 - **Pros:** ideal for *intra-query balancing*
 - **Pros:** new nodes can be added to the cluster without requiring heavy data redistribution
 - **Cons:** range queries become inefficient when *inter-query balancing* is the goal
 - **Pros/Cons:** depending on the hash function new nodes can be added to the cluster without requiring heavy data redistribution. For example Consistent hashing avoid the problem, while $X \bmod N$ would redistribute all the keys if on node is added to the cluster (i.e. $\#nodes = N+1$)

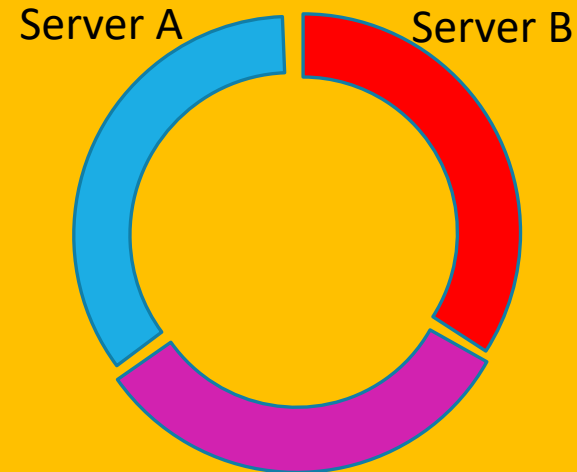
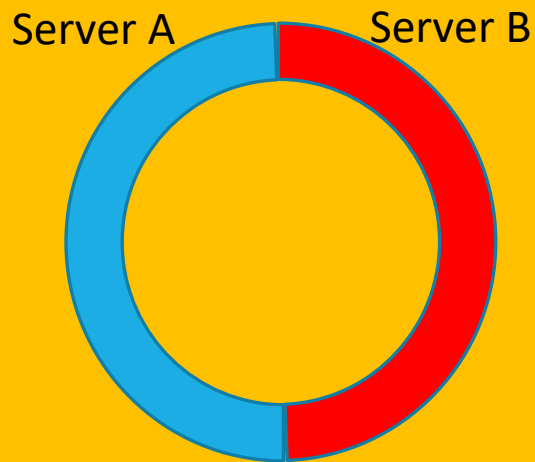
Sharding strategy

Server	x values		
A ($x \bmod 3 = 0$)	0	3	6
B ($x \bmod 3 = 1$)	1	4	7
C ($x \bmod 3 = 2$)	2	5	8

Server	x values		
A ($x \bmod 4 = 0$)	0	4	8
B ($x \bmod 4 = 1$)	1	5	
C ($x \bmod 4 = 2$)	2	6	
D ($x \bmod 4 = 3$)	3	7	

Sharding strategy

Assume data to be hashed to a position of a unit circle



Only the objects previously hashed to the violet region (1/3 of the whole) must be reassigned

Replication

Replication: the data is **copied** on several nodes

How to distribute the replicas?

- If the cluster is rack-aware, same strategy as for HDFS
 - One replica in the node receiving the data
 - One replica in a different node of another rack
 - One replica in a different node of that same rack

Each update must be pushed to every replica

Two techniques to handle updates:

- Master-slave
- Peer to peer

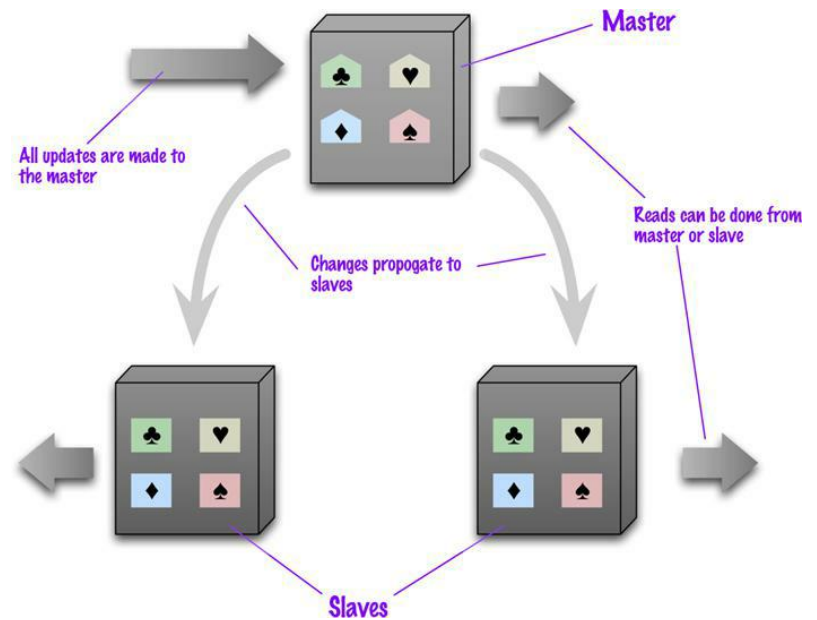
Master-Slave Replication

Master

- It's the manager of the data
- Handles each and every write operation
- Can be chosen or drawn

Slaves

- Enable read operations
- In sync with the master
- Can become master if the latter fails



Master-Slave Replication

Pros

- Easy handles many read requests
 - Slaves do not need the master to perform reads
- Useful when the workload mainly consists of reads

Cons

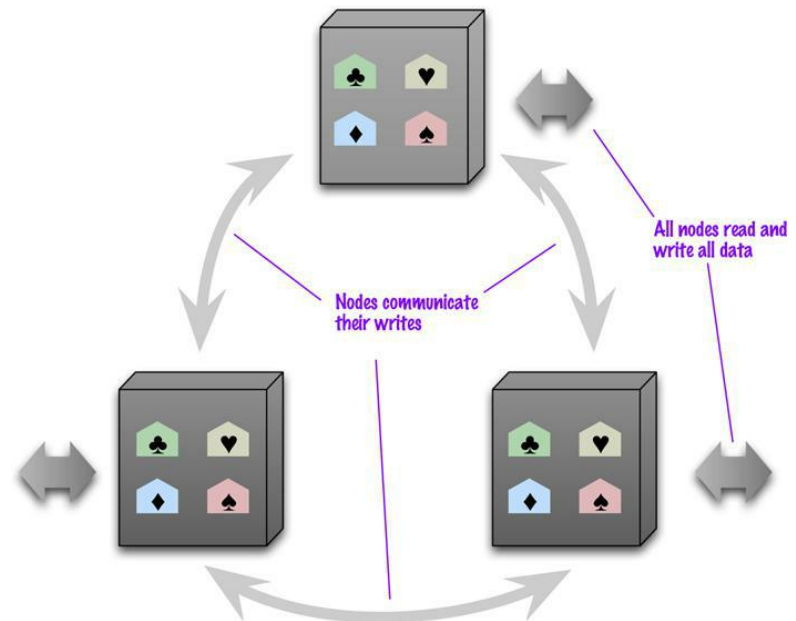
- **The master is a bottleneck**
 - **Only the master can handle writes**
 - In case of failure, a new master must be drawn
 - Not ideal when the workload mainly consists of write
- **Write propagation can be a source of inconsistency**
 - Two users may read different values at the same time
 - Read inconsistency can be problematic, but are relatively limited in time
 - The same problem occurs with peer-to-peer replication

Peer-to-Peer Replication

Each node has the same importance

Each node can handle write operations

The loss of a node does not compromise reads nor writes



Peer-to-Peer Replication

Pro

- The failure of a node does not interrupt read nor write requests
- Write performances easily scale by adding new nodes

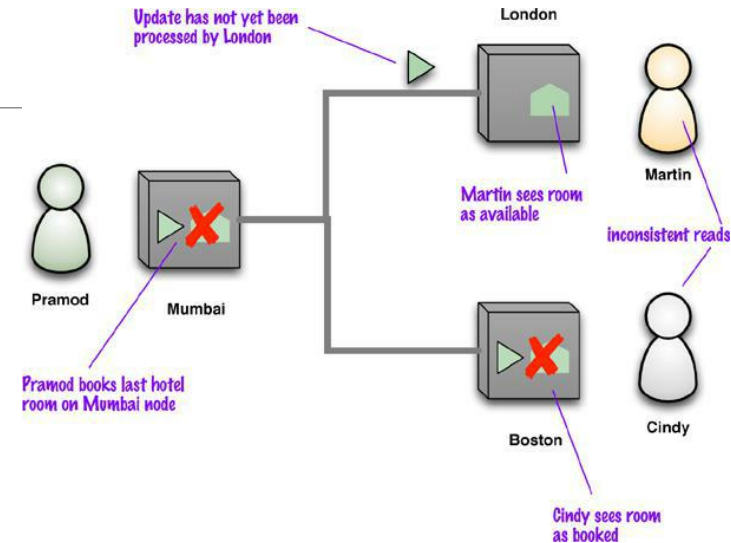
Cons

- **Write propagation can be a source of inconsistency**
 - Two users may read different values at the same time
 - Read inconsistency can be problematic, but are relatively limited in time
 - The same problem occurs with master-slave replication
- **Two users may update the same object in different replicas**
 - Write inconsistencies are way worse

Source of Inconsistency under Replication

Delays in updates

- Write propagation is not immediate
- Two users can access different replicas finding different values
- 1 update several reads or concurrent updates
- Take place both in the master-slave and peer-to-peer models

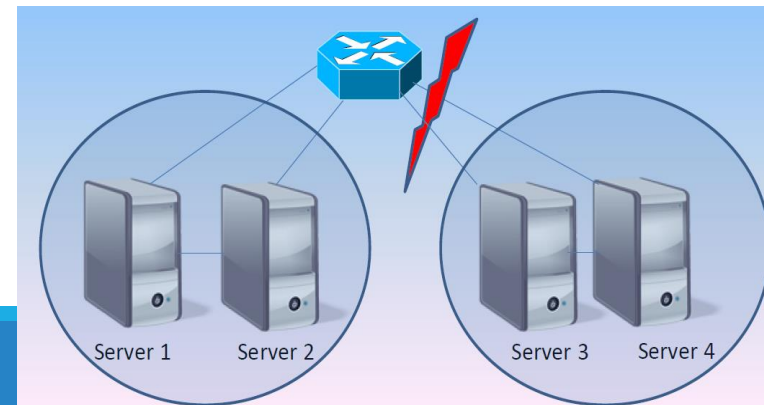


Network partitioning

- Due to network errors nodes in different partitions cannot communicate or some nodes may not be available
- Each group must decide independently whether it is allowed to carry out operations.

Applications that update many records

- ... when the system ensures consistency for single records only (no ACID properties)



Managing consistency

A LOOK BEHIND THE CURTAIN

A solid blue horizontal bar spanning the width of the slide at the bottom.

RDBMS vs NoSQL: different philosophies

RDBMSes come from decades of widespread usage

- Strong focus on data consistency
- Years of research activities to optimize performances
- Highly complex systems (triggers, caching, security, etc.)

NoSQL systems are designed to succeed where RDBMSes fail

- Strong focus on short latency and high availability
- Currently, quite simple systems
- Speed and manageability rather than consistency at all costs

RDBMS vs NoSQL: different philosophies

RDBMS pros

- Consistency and concurrency handled by the DBMS
- Trustworthy and sound security policies and mechanisms
- SQL standard: same code, different technologies
- Schema and constraints guarantee data quality
- Well known ER model and SQL
- Immediate integration with analytical tools

NoSQL pros

- Schema definition is not necessary to begin working
- Efficient data sharding
- Linear scalability with respect to the number of nodes in the cluster
- (Possibly) integrated text-search capabilities
- Freedom from ORM frameworks
- Easy management of high-variety data

Consistency: an example

Consider 1000€ to be transferred from bank account A to B; the transfer is made by:

- Removing 1000€ from A
- Adding 1000€ to B

What should never happen

- The money is removed from A but not added to B
- The money is added twice to B
- A query on the database shows an intermediate state
 - E.g., $A+B = 0\text{€}$

RDBMSes adopt **transactions** to avoid this kind of issue

Consistency in RDBMSs: ACID

Multi-operations transactions guarantee four fundamental properties: ACID

Atomicity

- The transaction is indivisible: either it fully completes, or it fails
- It cannot be completed partially

Consistency

- The transaction leaves the DB in a consistent state
- Integrity constraints can never be violated

Isolation

- The transaction is independent from the others
- In case of concurrent transactions, the effect is the same of their sequential execution

Durability

- The DBMS protects the DB from failures

Consistency in RDBMSs: ACID

Implementation of ACID properties relies on a **locking mechanisms and logs**

- Resources are locked, updates are logged
- In case of problems, rollback to the original state
- If no error occurs, unlock the resources

Consistency is guaranteed to the detriment of speed and availability

- User may have to wait
- Hard to replicate this mechanism in a distributed environment

But, sometimes, consistency is not that important

- E.g.: e-commerce application
- Shopping cart management requires speed and availability
- Order emission requires consistency

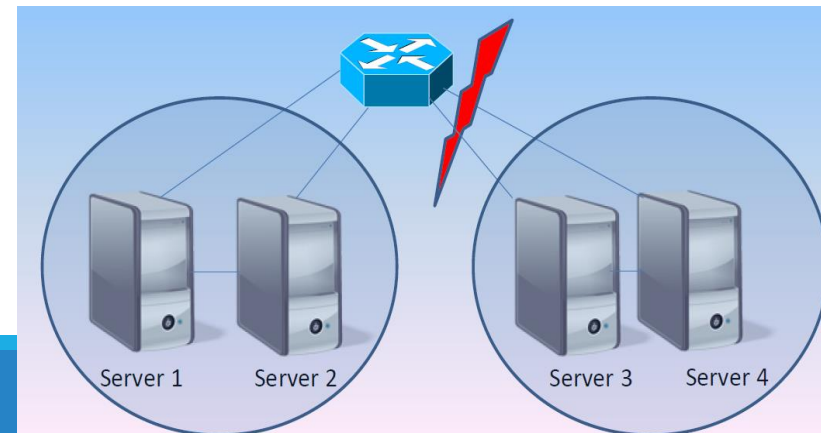
Consistency in NoSQL

Several attempts have been made to *describe* NoSQL properties with respect to ACID properties

- CAP theorem
- BASE properties
- PACELC theorem

The main cause of complexity increase in distributed databases is **Network partitioning** that happens when a network failure causes the isolation of a part of nodes in a collection.

- It can occur within a datacenter or between datacenters
- An arbitrary number of messages is lost





Prof. Eric A. Brewer
(Berkeley, Google)

Consistency in NoSQL: CAP

"Theorem": only two of the following three properties can be guaranteed

Consistency: every node in a distributed cluster returns the same, most recent, successful write. Consistency refers to every client having the same view of the data.

- There are various types of consistency models. Consistency in CAP (used to prove the theorem) refers to linearizability or sequential consistency, a very strong form of consistency.

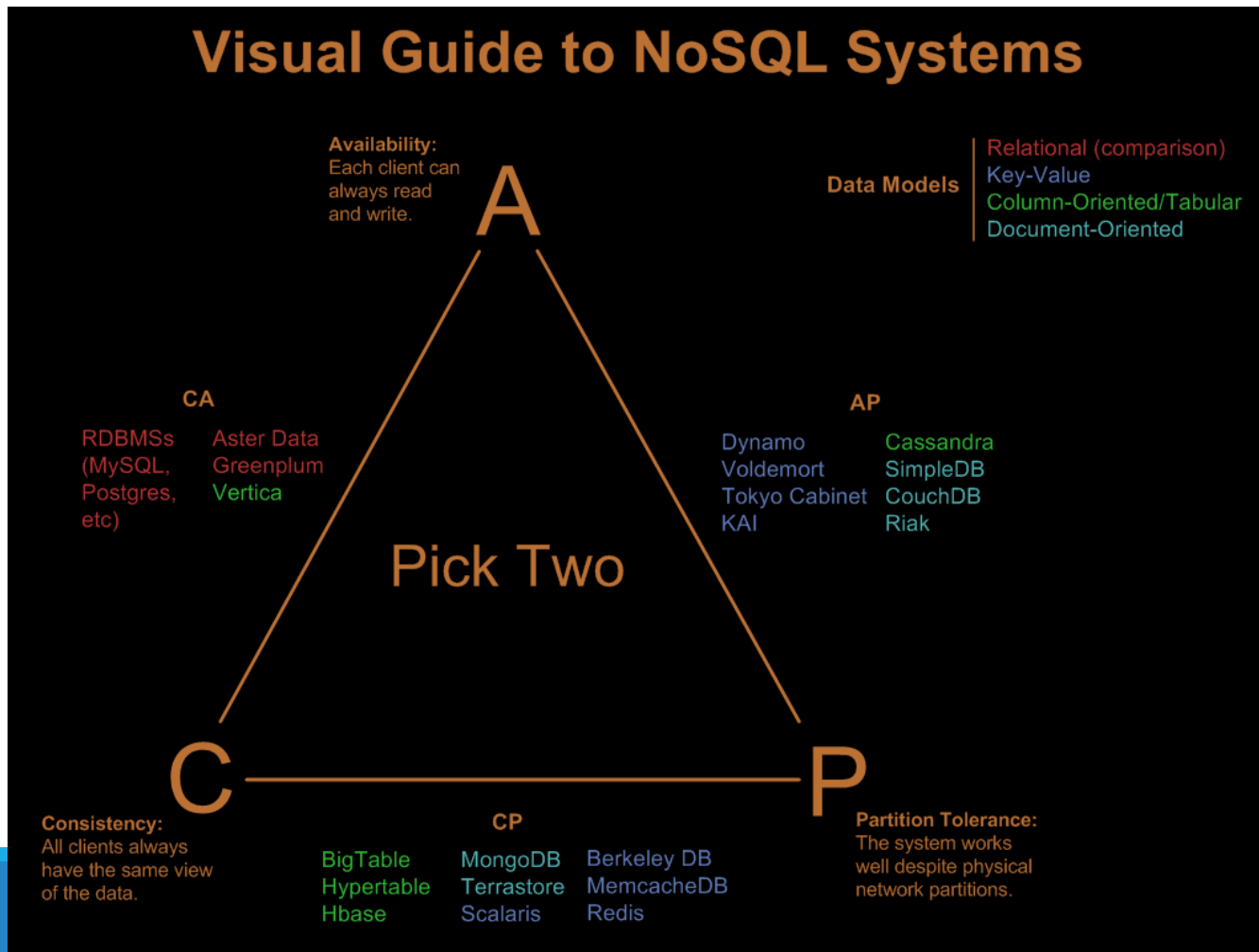
Availability: Every non-failing node returns a response for all read and write requests in a reasonable amount of time. The key word here is every. To be available, every node on (either side of a network partition) must be able to respond in a reasonable amount of time.

Partition tolerance: The system continues to function and upholds its consistency guarantees in spite of network partitions. Network partitions are a fact of life. Distributed systems guaranteeing partition tolerance can gracefully recover from partitions once the partition heals.

Theorem interpretation is not trivial

- Asymmetric properties: consistency is sacrificed to favor speed at all times, not just when partitioning happens
- Partition tolerance, consistency and availability behaves differently based on how strict is the requisite and the algorithms implementing them

Consistency in NoSQL: CAP



Consistency in NoSQL: CAP



Prof. Eric A. Brewer
(Berkeley, Google)

Three situations

- CA: the system cannot suffer from network partitioning (single server)
- AP: in case of partitioning, the system sacrifices consistency (overbooking)
- CP: in case of partitioning, the system sacrifices availability (bookings prevented)
- Since different DBMSs make different choices, choosing the right DBMS depends on the application constraints:

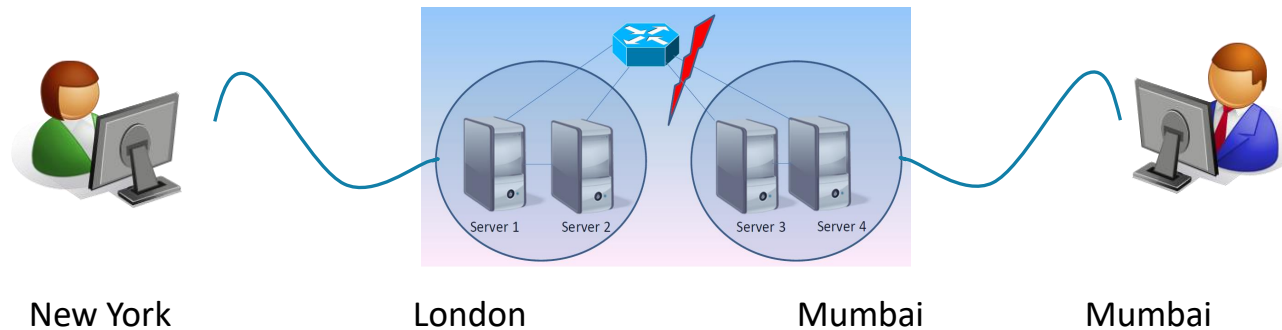
When network partitioning is plausible:

DBMSes enforcing CP: when it is imperative that all clients have a consistent view of the database, users of a node will have to wait for any other node to agree before they can read or write to the database, availability is affected since users have to wait an unspecified period of time. HBase supports such CP's model

DBMSes enforcing AP: when the database must remain available at all times, DBMSs that allow clients to write data to a node in the database without waiting for other nodes to agree. The DBMS then takes care of the reconciliation of the data at a later time. This is the final state of consistency. In applications that could sacrifice data consistency in exchange for huge performance, databases such as CouchDB, Cassandra can be selected.

An Example

- Ann is trying to book a room of the Ace Hotel in New York on a node located in London of a booking system
- Pathin is trying to do the same on a node located in Mumbai
- The booking system uses a peer-to-peer distribution
- There is only a room available
- The network link breaks



Possible Solutions

CA: Neither user can book any hotel room

CP: Designate Mumbai node as the master for Ace hotel

- Pathin can make the reservation
- Ann can see the inconsistent room information
- Ann cannot book the room

AP: both nodes accept the hotel reservation

- Overbooking!
- These situations are closely tied to the domain
 - Financial exchanges? Blogs? Shopping charts?
 - An hotel could regularly accept overbooking aware that a percentage of customers cancel
- Issues
 - How tolerant you are of stale reads
 - How long the inconsistency window persist

Common approach:

- BASE (Basically Available, Soft state, Eventual consistency)

Consistency in NoSQL: BASE

The CAP theorem is often cited as a justification for the use of weaker consistency models, for example **BASE** (Basically Available Soft-state (services with) Eventual consistency).

Basic **A**vailability

- The system should always be available

Soft-state

- It is acceptable for the system to be temporarily inconsistent

Eventual consistency

- Eventually, the system becomes consistent
- The most important property

Consistency in NoSQL: BASE

ACID

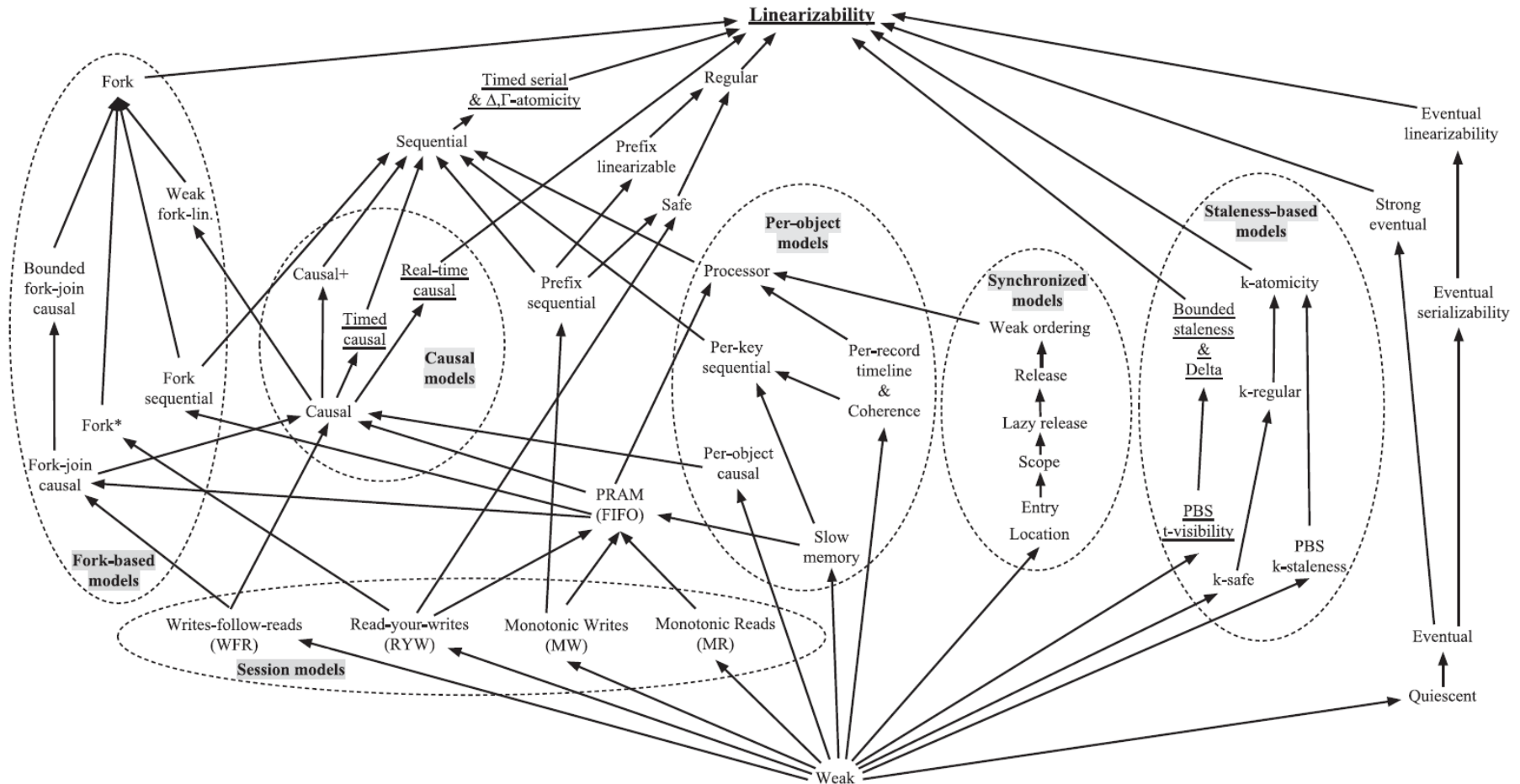
- Pessimistic approach (better safe than sorry)

BASE

- Optimistic approach (everything is going to be ok)
- Higher throughput is better than enforcing consistency

In summary, the BASE model is characterized by a high availability for first level services; a cleaning mechanism is adopted to solve the problems created by optimistic actions which then appear to have violated the consistency.

Consistency Models in Non-Transactional Distributed Storage Systems



Forms of Consistency

In the following we refer to single-record transactions in contrast to multi-record transactions considered in RDBMS

- Strong consistency
- Sequential consistency
- Session (or read-your-writes) or causal consistency
- Eventual consistency

Strong Consistency

Strong Consistency or Linearization is the strongest consistency model. Each operation must appear committed immediately, and all clients will operate over the same data state. A read operation in an object must wait until the write commits before being able to read the new version. There is also a single global order of events accepted by all storage systems' instances.

Strong Consistency leads to a high consistency system, but it compromises scaling by decreasing availability and network partition tolerance

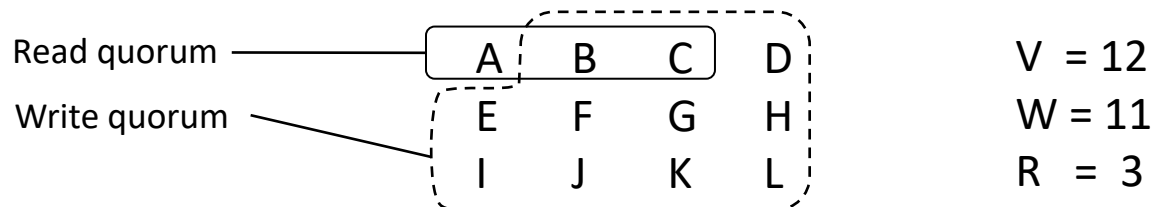
The quorum mechanism

The **quorum mechanism** can ensure strong consistent reads and writes under replication

- Based on contacting a majority of the nodes responsible for certain data
- Each copy of a replicated data item is assigned a vote.
- Each operation has to obtain a *read quorum* (R) or a *write quorum* (W) to read or write a data item, respectively.
- If a given data item has a total of V votes, the quorums have to obey the following rules:

Given a data item with V votes (i.e. number of replicas), proper quorums must obey the following rules:

- **Writing quorum:** $W > V/2$
 - Ensures that two write operations from two transactions cannot occur concurrently on the same data item.
- **Reading quorum:** $R + W > V$
 - Ensures that a data item is not read and written by two transactions concurrently



The Cassandra Example

In Cassandra, a Consistency Level (CL) is the number of replicas responding before returning a reply to a user.

- For read operations, the consistency level specifies how many replica nodes must respond with the latest consistent data before the coordinator successfully sends the data back to the client.
- For write operations, the consistency level specifies how many replica nodes must acknowledge back before the coordinator successfully reports back to the client.

CL is tunable for each single read and write

- $R = CL$ for reads
- $W = CL$ for writes
- $RF =$ Replication factor

Varying the CLs determines a trade-off between consistency and availability

- $R + W > RF$ implies strong consistency

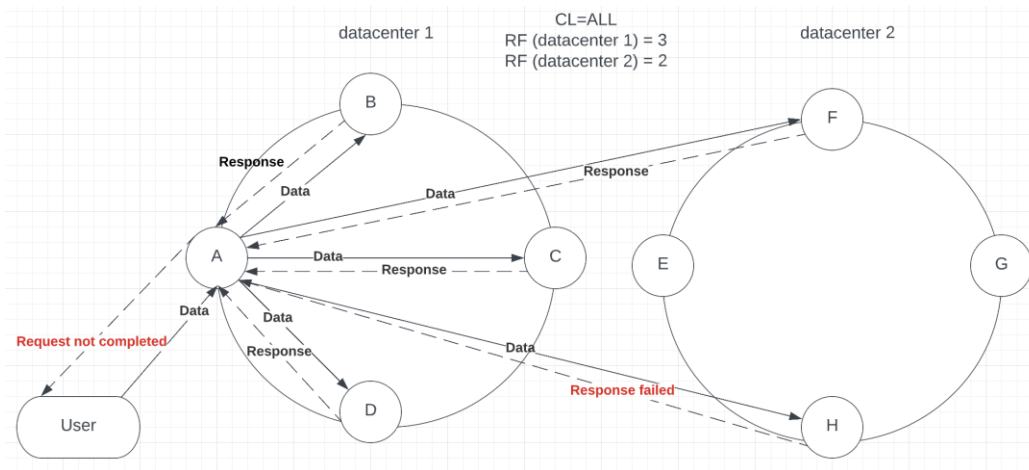
The Cassandra Example

In Cassandra Consistency Level (CL) is tunable

Level	Description	Usage	Level	Description	Usage
ALL	A write must be written to the commit log and memtable on all replica nodes in the cluster for that partition.	Provides the highest consistency and the lowest availability of any other level.	ALL	Returns the record after all replicas have responded. The read operation will fail if a replica does not respond.	Provides the highest consistency of all levels and the lowest availability of all levels.
EACH_QUORUM	Strong consistency. A write must be written to the commit log and memtable on a quorum of replica nodes in <i>each</i> datacenter .	Used in multiple datacenter clusters to strictly maintain consistency at the same level in each datacenter. For example, choose this level if you want a write to fail when a datacenter is down and the QUORUM cannot be reached on that datacenter.	QUORUM	Returns the record after a quorum of replicas from all datacenters has responded.	Used in either single or multiple datacenter clusters to maintain strong consistency across the cluster. Ensures strong consistency if you can tolerate some level of failure.
QUORUM	A write must be written to the commit log and memtable on a quorum of replica nodes across <i>all</i> datacenters.	Used in either single or multiple datacenter clusters to maintain strong consistency across the cluster. Use if you can tolerate some level of failure.	LOCAL_QUORUM	Returns the record after a quorum of replicas in the current datacenter as the coordinator has reported. Avoids latency of inter-datacenter communication.	Used in multiple datacenter clusters with a rack-aware replica placement strategy (Network Topology Strategy) and a properly configured snitch. Fails when using SimpleStrategy.
LOCAL_QUORUM	Strong consistency. A write must be written to the commit log and memtable on a quorum of replica nodes in the same datacenter as the coordinator . Avoids latency of inter-datacenter communication.	Used in multiple datacenter clusters with a rack-aware replica placement strategy, such as NetworkTopologyStrategy , and a properly configured snitch. Use to maintain consistency locally (within the single datacenter). Can be used with SimpleStrategy .	ONE	Returns a response from the closest replica, as determined by the snitch . By default, a read repair runs in the background to make the other replicas consistent.	Provides the highest availability of all the levels if you can tolerate a comparatively high probability of stale data being read. The replicas contacted for reads may not always have the most recent write.
ONE	A write must be written to the commit log and memtable of at least one replica node.	Satisfies the needs of most users because consistency requirements are not stringent.			

The Cassandra Example

If CL is set to all....



Session Consistency

Inconsistency windows can be particularly problematic when you get inconsistencies with yourself.

Read-your-writes consistency means that, once you've made an update, you're guaranteed to continue seeing that update.

- There are situations where guaranteeing this may not be so obvious: consider the example of posting comments on a blog entry. Often, systems handle the load of such sites by running on a cluster and load-balancing incoming requests to different nodes. This means that you may post a message using one node, then refresh your browser, but the refresh goes to a different node which hasn't received your post yet!

Session consistency guarantees **read-your-writes consistency** within a session: if the session ends for some reason, or the user access simultaneously the same system from different computers, she may lose consistency.

Session Consistency

There are a couple of techniques to guarantee session consistency.

- **Sticky session:** a session that's tied to one node (this is also called session affinity). The downside is that sticky sessions reduce the ability of the load balancer to do its job.
- **Version stamps:** ensuring that every interaction with the data store includes the latest version stamp seen by a session. The server node must then ensure that it has the updates that include that version stamp before responding to a request.
 - A version stamp is a field that changes every time the data in the database changes. When you read the data you keep a note of the version stamp, so that when you write data you can check to see if the version has changed.

Eventual Consistency

Replicas tend to converge to the same data state. While this convergence process runs, it is possible for read operations to retrieve an older version instead of the latest one. The *inconsistency window* will depend on communication delays between replicas and its sources, the load on the system and the number of replicas involved